Instituto Profesional Duoc UC.
Escuela de Informática y Telecomunicaciones
Sede: Plaza Norte

“SaludConex: Sistema de Predicción de Inasistencias Médicas”

Bastian Ignacio Roman Fuenzalida

Ignacio Andres Gomez Olate

Vicente Contreras

Sección: 001-D

Nombre del docente: Osnellys E, Andrade D.

ÍNDICE

1. Introducción
2. Contexto y descripción del proyecto
3. Objetivos de la implementación
4. Desarrollo técnico de la solución
   4.1. Comprensión y exploración de los datos
   4.2. Preparación y transformación de los datos
   4.3. Desarrollo del modelo predictivo
   4.4. Evaluación del modelo
   4.5. Generación de archivos para Business Intelligence
5. Interpretación general de los resultados
6. Desarrollo del dashboard en Looker Studio
   6.1. Objetivo del dashboard
   6.2. Estructura del dashboard
   6.3. Visualizaciones e indicadores
   6.4. Evidencias del dashboard
   6.5. Integración de los datos
7. Gestión, seguimiento y control del proyecto
8. Desviaciones, dificultades y acciones correctivas
9. Limitaciones y consideraciones éticas
10. Conclusiones
11. Anexos

1. INTRODUCCIÓN

El presente informe documenta el desarrollo técnico del proyecto SaludConex, cuyo propósito es construir una solución de ciencia de datos que permita anticipar posibles inasistencias a consultas médicas.

La iniciativa surge frente a una problemática habitual en los centros de atención: cuando un paciente no asiste a una hora reservada y no informa previamente, se pierde capacidad de atención, se desaprovechan recursos y otros pacientes pueden enfrentar mayores tiempos de espera.

Para abordar esta situación, se desarrolló un prototipo predictivo utilizando herramientas de programación y aprendizaje automático. El trabajo contempla la exploración de datos, su preparación, el entrenamiento y evaluación de distintos modelos de clasificación, y la generación de archivos que permiten trasladar los resultados hacia una herramienta de inteligencia de negocios.

El desarrollo se organizó siguiendo las etapas de la metodología CRISP-DM, lo que permitió avanzar desde la comprensión del problema hasta la preparación de los resultados para su visualización. Python fue utilizado para el procesamiento de los datos y la construcción del modelo, mientras que Looker Studio se considera la herramienta destinada a presentar los resultados mediante un dashboard interactivo.

Es importante señalar que, debido a que no se dispuso de registros históricos reales de SaludConex, se trabajó con un conjunto de datos sintético de 50.000 reservas. Por lo tanto, los resultados obtenidos corresponden a una simulación académica y no representan el comportamiento real de pacientes o centros médicos de la institución.

2. CONTEXTO Y DESCRIPCIÓN DEL PROYECTO

SaludConex es un escenario de una red privada de atención médica que busca mejorar la gestión de sus reservas. Dentro del contexto planteado para el proyecto, la organización cuenta con ocho centros médicos y aproximadamente 32.000 atenciones mensuales. Se establece como antecedente una tasa de inasistencia cercana al 17%, situación que puede generar horas médicas desaprovechadas y dificultades para administrar la disponibilidad de atención.

Actualmente, el proceso de recordatorios considera el envío de avisos a los pacientes antes de sus consultas. Sin embargo, realizar esta acción de manera general no permite distinguir cuáles reservas podrían presentar un mayor riesgo de inasistencia.

A partir de esta necesidad, se propone desarrollar un modelo que analice características de las reservas y antecedentes de comportamiento para generar un puntaje que permita identificar los casos que podrían requerir una revisión preventiva.

La solución no busca reemplazar las decisiones del personal administrativo ni cancelar automáticamente las consultas. Su finalidad es entregar información adicional para apoyar la gestión y facilitar que los equipos puedan priorizar sus acciones de contacto.

3. OBJETIVOS DE LA IMPLEMENTACIÓN

3.1. Objetivo general

Desarrollar un prototipo de ciencia de datos capaz de estimar el riesgo de inasistencia de una reserva médica, utilizando técnicas de aprendizaje automático y herramientas de visualización que permitan interpretar los resultados y apoyar la gestión preventiva de SaludConex.

3.2. Objetivos específicos

- Explorar y analizar el conjunto de datos para comprender su estructura, calidad y comportamiento.
- Preparar las variables necesarias para desarrollar un modelo de clasificación.
- Comparar distintos algoritmos de aprendizaje automático y seleccionar el que presente mejores resultados para el objetivo del proyecto.
- Evaluar el desempeño del modelo mediante métricas de clasificación y análisis de errores.
- Generar archivos estructurados con las predicciones y los resultados de evaluación.
- Construir un dashboard en Looker Studio que permita consultar e interpretar la información generada.
- Documentar las decisiones técnicas, las limitaciones y los resultados obtenidos durante la implementación.

4. DESARROLLO TÉCNICO DE LA SOLUCIÓN

El desarrollo técnico se realizó mediante Python, utilizando principalmente bibliotecas para manipulación de datos, visualización y aprendizaje automático. El proceso se organizó en

distintas etapas para mantener una secuencia clara entre la exploración, preparación, modelamiento y evaluación.

4.1. Comprensión y exploración de los datos

La primera etapa consistió en revisar el conjunto de datos y conocer las características de la información disponible. Para esto se trabajó con un dataset sintético de 50.000 registros y 18 variables, que representan información relacionada con las reservas médicas, los pacientes y el resultado de las atenciones.

Entre las variables disponibles se encuentran la edad, comuna, centro médico, especialidad, fecha de solicitud, fecha de atención, días de anticipación, historial de asistencias, cancelaciones e inasistencias previas, además del estado final de la reserva.

Se utilizaron herramientas de Pandas para revisar las dimensiones del conjunto de datos, los tipos de variables y sus estadísticas descriptivas. También se verificó la presencia de valores faltantes y registros duplicados, con el propósito de conocer las condiciones iniciales antes de comenzar el procesamiento.

En esta revisión se identificaron 50.000 registros, sin valores nulos ni reservas duplicadas según las comprobaciones realizadas en el notebook. Esto permitió continuar con el análisis sin necesidad de aplicar una eliminación inicial de registros por problemas de integridad.

Posteriormente, se estudió la distribución de los resultados de las reservas y se compararon las tasas de inasistencia según distintas características.

Análisis por especialidad

Se observaron diferencias entre especialidades. Traumatología presentó una tasa de inasistencia del 17,66%, seguida de Kinesiología con un 17,56%. En cambio, Pediatría registró un 15,06%.

Estas diferencias permiten identificar variaciones dentro del conjunto simulado, aunque no son suficientes para afirmar que la especialidad sea la causa de una inasistencia.

Análisis por centro médico

Al comparar los centros, Ñuñoa presentó una tasa de inasistencia del 18,55%, mientras que La Florida registró un 16,26%. Las diferencias no fueron extremas y los valores se mantuvieron aproximadamente entre el 16% y el 19%.

Análisis por edad

La exploración mostró que el grupo de 75 años o más presentó una tasa de inasistencia del 18,23%. También se observaron valores relativamente elevados en los grupos de 60 a 74 años y de 0 a 17 años.

Análisis por días de anticipación

Una de las observaciones más relevantes apareció al revisar la cantidad de días entre la solicitud de la reserva y la fecha de atención. El grupo de reservas realizadas con una anticipación de 61 a 90 días presentó una tasa de inasistencia del 34,43%.

Sin embargo, este último resultado debe interpretarse con precaución, ya que dicho segmento contiene solamente 61 registros. Un volumen tan reducido puede provocar variaciones importantes en el porcentaje observado.

En general, la exploración permitió reconocer que existen diferencias entre grupos y características de las reservas que pueden ser útiles para construir un modelo predictivo. También permitió comprender que los porcentajes observados representan asociaciones dentro de los datos simulados y no relaciones de causalidad.

01. Estructura del conjunto de datos

Fuente: SaludConex(7).ipynb · Celda(s): 7

CÓDIGO DEL NOTEBOOK

```
CELDA 7
df.info()
```

SALIDA GUARDADA

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 50000 entries, 0 to 49999
Data columns (total 18 columns):
 #   Column                      Non-Null Count  Dtype
---  ------                      --------------  -----
 0   ID_Reserva                  50000 non-null  object
 1   ID_Paciente_Anonimizado     50000 non-null  object
 2   Edad                        50000 non-null  int64
 3   Comuna                      50000 non-null  object
 4   Centro_Medico               50000 non-null  object
 5   Especialidad                50000 non-null  object
 6   Profesional_ID              50000 non-null  object
 7   Fecha_Solicitud             50000 non-null  datetime64[ns]
 8   Fecha_Atencion              50000 non-null  datetime64[ns]
 9   Horario_Atencion            50000 non-null  object
 10  Dias_Anticipacion           50000 non-null  int64
 11  Asistencias_Previas         50000 non-null  int64
 12  Cancelaciones_Previas       50000 non-null  int64
 13  Inasistencias_Previas       50000 non-null  int64
 14  Tipo_Previsión              50000 non-null  object
 15  Modalidad_Atencion          50000 non-null  object
 16  Confirmacion_Recordatorio   50000 non-null  object
 17  Resultado_Reserva           50000 non-null  object
dtypes: datetime64[ns](2), int64(5), object(11)
memory usage: 6.9+ MB
```

Salida de df.info(): 50.000 registros, 18 columnas y tipos de datos.

02. Estadísticas descriptivas

Fuente: SaludConex(7).ipynb · Celda(s): 9

	count	mean	std	min	25%	50%	75%	max
Edad	50000.0	45.39220	25.722391	1.0	23.0	46.0	68.0	89.0
Dias_Anticipacion	50000.0	12.21330	9.553003	0.0	5.0	10.0	17.0	90.0
Asistencias_Previas	50000.0	4.00750	2.001410	0.0	3.0	4.0	5.0	16.0
Cancelaciones_Previas	50000.0	1.04410	1.047176	0.0	0.0	1.0	2.0	9.0
Inasistencias_Previas	50000.0	0.94408	1.040968	0.0	0.0	1.0	1.0	8.0

Tabla de resumen de las variables numéricas.

03. Revisión de valores faltantes

Fuente: SaludConex(7).ipynb · Celda(s): 10

	Valores faltantes	Porcentaje (%)
ID_Reserva	0	0.0
ID_Paciente_Anonimizado	0	0.0
Edad	0	0.0
Comuna	0	0.0
Centro_Medico	0	0.0
Especialidad	0	0.0
Profesional_ID	0	0.0
Fecha_Solicitud	0	0.0
Fecha_Atencion	0	0.0
Horario_Atencion	0	0.0
Dias_Anticipacion	0	0.0
Asistencias_Previas	0	0.0
Cancelaciones_Previas	0	0.0
Inasistencias_Previas	0	0.0
Tipo_Previsión	0	0.0
Modalidad_Atencion	0	0.0
Confirmacion_Recordatorio	0	0.0
Resultado_Reserva	0	0.0

Salida de la comprobación de valores faltantes por variable.

04. Conteo de resultados de las reservas

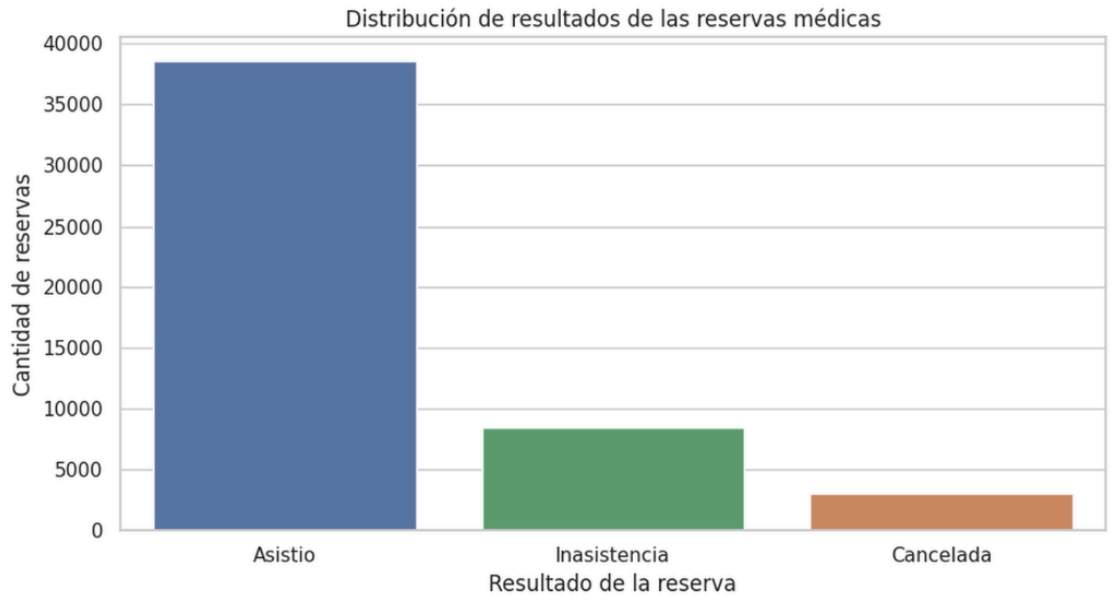
Fuente: SaludConex(7).ipynb · Celda(s): 11

Resultado_Reserva	Cantidad	Porcentaje (%)
Asistio	38562	77.12
Inasistencia	8450	16.90
Cancelada	2988	5.98

Cantidad y porcentaje de asistencias, inasistencias y cancelaciones.

Gráfico: distribución de resultados de las reservas

Fuente: SaludConex(7).ipynb · Celda 12



05. Revisión de duplicados e inconsistencias

Fuente: SaludConex(7).ipynb · Celda(s): 16

CÓDIGO DEL NOTEBOOK

CELDA 16

```
print("Registros completamente duplicados:", df.duplicated().sum())

# Revisar identificadores de reservas duplicados
print("Reservas con ID duplicado:", df['ID_Reserva'].duplicated().sum())

# Revisar inconsistencias
print("Edades fuera de rango:",
      ((df['Edad'] < 0) | (df['Edad'] > 100)).sum())

print("Anticipaciones negativas:",
      (df['Dias_Anticipacion'] < 0).sum())

print("Fechas incorrectas:",
      (df['Fecha_Solicitud'] > df['Fecha_Atencion']).sum())
```

SALIDA GUARDADA

```
Registros completamente duplicados: 0
Reservas con ID duplicado: 0
Edades fuera de rango: 0
Anticipaciones negativas: 0
Fechas incorrectas: 0
```

Salida de controles de duplicados, edades, anticipaciones y fechas.

06. Tasa de inasistencia por especialidad

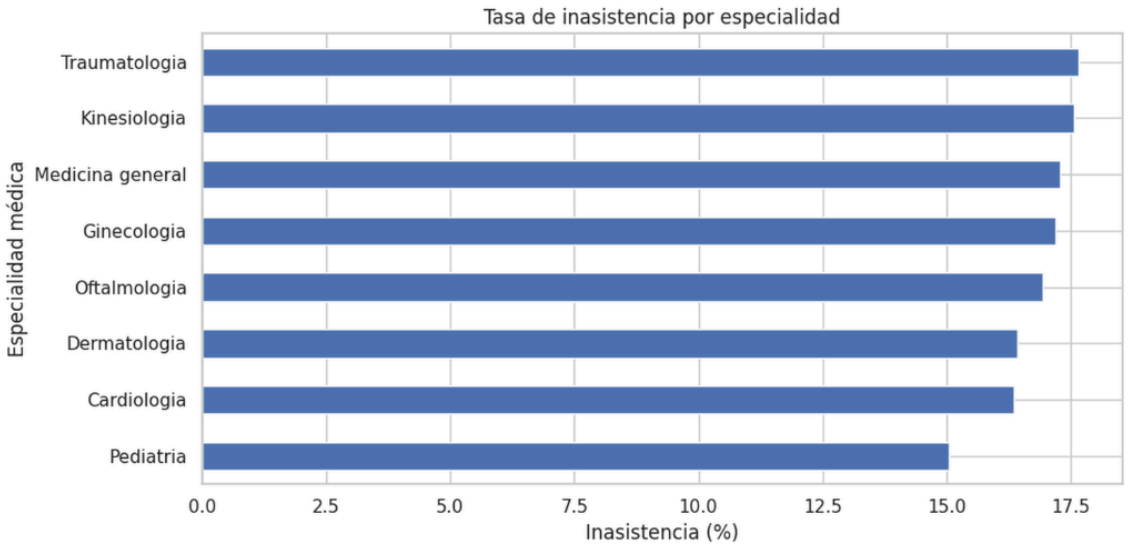
Fuente: SaludConex(7).ipynb · Celda(s): 20

```
Especialidad
Traumatologia       17.663005
Kinesiologia        17.563336
Medicina general    17.289193
Ginecologia         17.204478
Oftalmologia        16.942384
Dermatologia        16.432763
Cardiologia         16.357710
Pediatria           15.057529
Name: Es_Inasistencia, dtype: float64
```

Tabla de tasas calculadas a partir del conjunto sintético.

Gráfico: tasa de inasistencia por especialidad

Fuente: SaludConex(7).ipynb · Celda 21



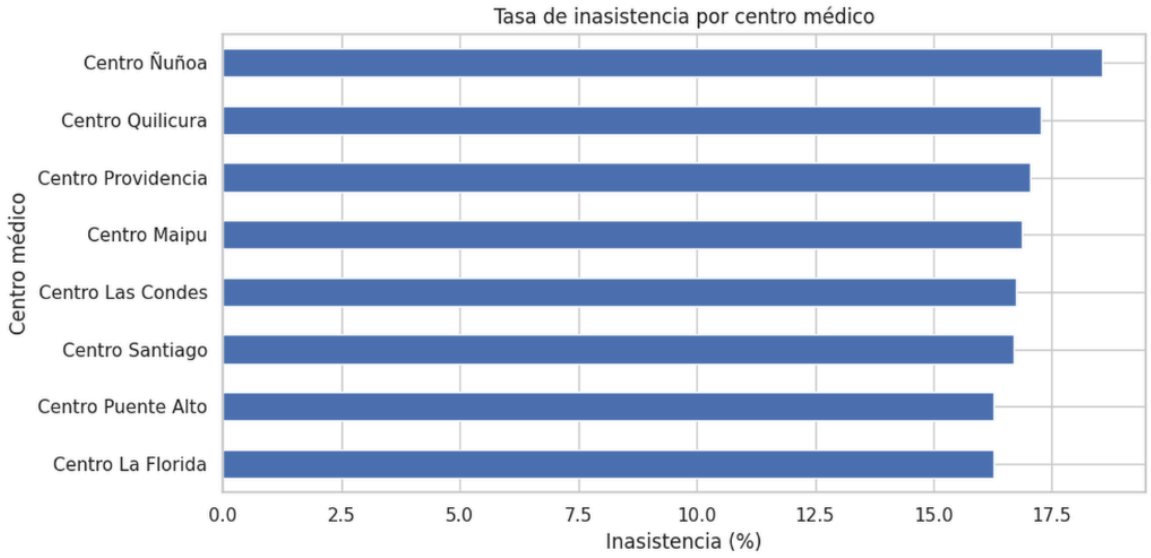
07. Tasa de inasistencia por centro médico

Fuente: SaludConex(7).ipynb · Celda(s): 23

```
Centro_Medico
Centro Ñuñoa          18.552876
Centro Quilicura      17.262144
Centro Providencia    17.045132
Centro Maipu          16.858742
Centro Las Condes     16.740823
Centro Santiago       16.701211
Centro Puente Alto    16.273312
Centro La Florida     16.259530
Name: Es_Inasistencia, dtype: float64
```

Gráfico: tasa de inasistencia por centro médico

Fuente: SaludConex(7).ipynb · Celda 24



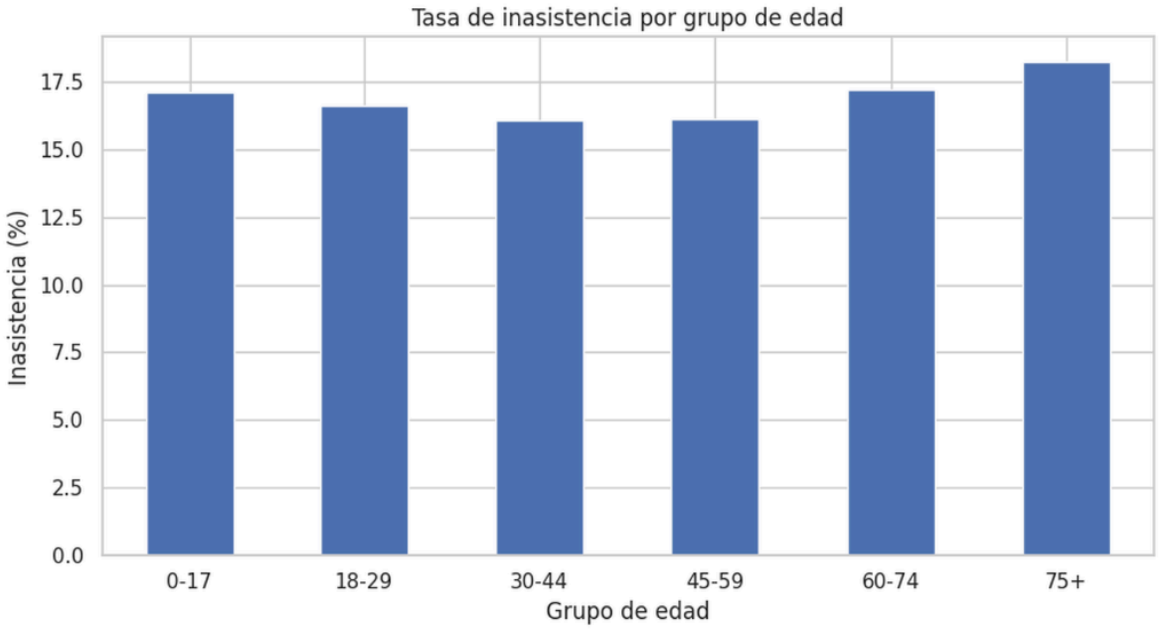
08. Tasa de inasistencia por grupo de edad

Fuente: SaludConex(7).ipynb · Celda(s): 26

```
Grupo_Edad
0-17     17.109948
18-29    16.602495
30-44    16.043354
45-59    16.095398
60-74    17.166725
75+      18.226657
Name: Es_Inasistencia, dtype: float64
```

Gráfico: tasa de inasistencia por grupo de edad

Fuente: SaludConex(7).ipynb · Celda 27



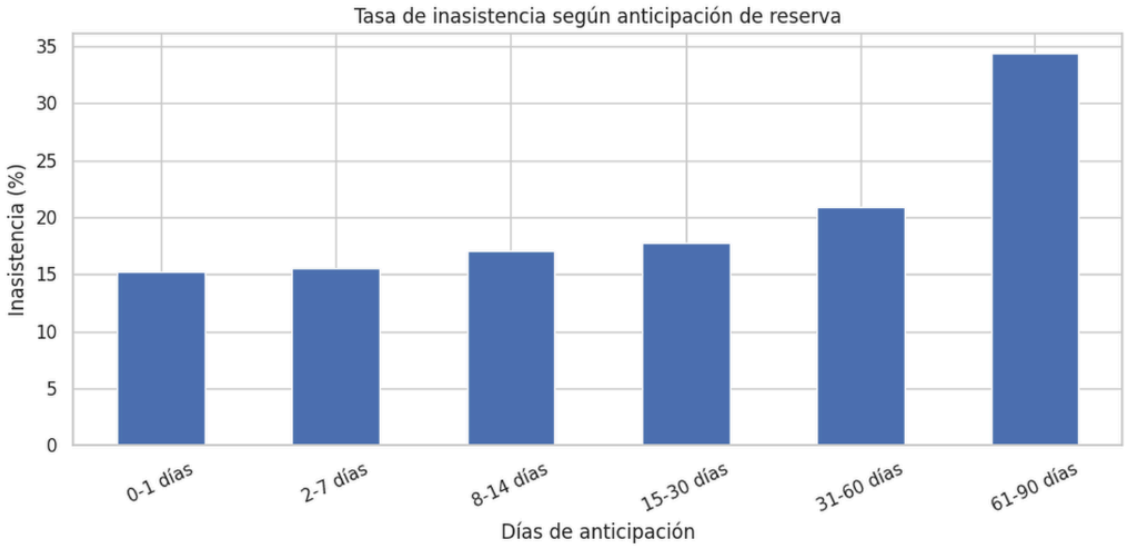
09. Tasa de inasistencia por días de anticipación

Fuente: SaludConex(7).ipynb · Celda(s): 29

```
Grupo_Anticipacion
0-1 días      15.195312
2-7 días      15.579866
8-14 días     17.101262
15-30 días    17.765876
31-60 días    20.905923
61-90 días    34.426230
Name: Es_Inasistencia, dtype: float64
```

Gráfico: tasa de inasistencia por anticipación

Fuente: SaludConex(7).ipynb · Celda 30



4.2. Preparación y transformación de los datos

Una vez finalizada la exploración, se realizó la preparación del conjunto de datos para que pudiera ser utilizado por los algoritmos de aprendizaje automático.

Primero se identificaron las variables que no debían participar directamente en el entrenamiento. Se excluyeron identificadores como **ID_Reserva** e **ID_Paciente_Anonimizado**, debido a que su función es identificar registros y no representar patrones generales de comportamiento. También se descartó **Profesional_ID** y se retiraron determinadas variables de fecha directa y agrupaciones utilizadas principalmente para el análisis exploratorio.

Otro paso importante fue separar las reservas canceladas de aquellas que terminaron en asistencia o inasistencia. Se eliminaron 2.988 reservas canceladas, debido a que el

objetivo del proyecto es anticipar las inasistencias que ocurren cuando el paciente no se presenta, no predecir cancelaciones informadas previamente.

A partir de esto se construyó la variable objetivo **Objetivo_Inasistencia**, que utiliza dos valores:

- 0: La reserva terminó en asistencia.
- 1: La reserva terminó en inasistencia.

Para el entrenamiento se seleccionaron 14 variables predictoras, divididas en cinco numéricas y nueve categóricas. Esta separación permitió aplicar tratamientos diferentes según la naturaleza de cada variable.

Se construyó un pipeline de preprocesamiento utilizando **ColumnTransformer**, **SimpleImputer**, **StandardScaler** y **OneHotEncoder**.

El tratamiento consideró la imputación mediante la mediana para variables numéricas y el valor más frecuente para las categóricas. Posteriormente, las variables numéricas fueron estandarizadas y las categóricas transformadas mediante codificación One-Hot, permitiendo que los algoritmos trabajaran con información numérica.

Como resultado, las 14 variables iniciales se transformaron en 64 columnas procesables por los modelos.

Prevención de fuga de información

Una decisión metodológica importante fue evitar el Data Leakage o fuga de información. Este problema ocurre cuando el modelo utiliza datos que no estarían disponibles al momento de realizar una predicción real, provocando que los resultados de evaluación sean artificialmente mejores.

Para reducir este riesgo, se excluyó la variable de resultado de la reserva y cualquier información derivada directamente de ella. Además, el preprocesador se ajustó únicamente con los datos de entrenamiento y luego se aplicó sobre los conjuntos posteriores mediante transform().

El uso de **Confirmacion_Recordatorio** se definió bajo un supuesto operativo específico: la predicción se realizaría después del envío del recordatorio de 24 horas, cuando ese dato ya estaría disponible.

Separación temporal de los datos

En lugar de realizar una división aleatoria, se utilizó una separación cronológica. Esta decisión permite simular de mejor manera el escenario en el que un modelo aprende a partir de información histórica y posteriormente predice sobre reservas más recientes.

La separación dejó los siguientes conjuntos:

- Entrenamiento: 37.569 registros.
- Prueba: 9.443 registros.
- Fecha de corte: 6 de abril de 2026.

La distribución de la variable objetivo fue similar en ambos conjuntos. En entrenamiento, las inasistencias representaron el 17,93%, mientras que en prueba alcanzaron el 18,13%.

Esta similitud permitió continuar con el modelamiento sin observar una diferencia importante en la proporción de clases entre ambos periodos.

10. Tratamiento de reservas canceladas

Fuente: SaludConex(7).ipynb · Celda(s): 40

CÓDIGO DEL NOTEBOOK

```
CELDA 40
canceladas = (
    df_model['Resultado_Reserva'] == 'Cancelada'
).sum()

# Conservar únicamente asistencias e inasistencias
df_model = df_model[
    df_model['Resultado_Reserva'].isin(
        ['Asistio', 'Inasistencia']
    )
].copy()

# Crear variable objetivo para el modelo
df_model['Objetivo_Inasistencia'] = (
    df_model['Resultado_Reserva'] == 'Inasistencia'
).astype(int)

print("Reservas canceladas excluidas:", canceladas)
print("Registros disponibles para modelamiento:", len(df_model))

# Revisar distribución de la variable objetivo
display(
    df_model['Objetivo_Inasistencia'].value_counts()
)
```

SALIDA GUARDADA

```
Reservas canceladas excluidas: 2988
Registros disponibles para modelamiento: 47012

Objetivo_Inasistencia
0    38562
1     8450
Name: count, dtype: int64
```

11. Selección de variables predictoras

Fuente: SaludConex(7).ipynb · Celda(s): 43

CÓDIGO DEL NOTEBOOK

CELDA 43

```
variables_numericas = [
    'Edad',
    'Dias_Anticipacion',
    'Asistencias_Previas',
    'Cancelaciones_Previas',
    'Inasistencias_Previas'
]

# Variables categóricas
variables_categoricas = [
    'Comuna',
    'Centro_Medico',
    'Especialidad',
    'Horario_Atencion',
    'Tipo_Previsión',
    'Modalidad_Atencion',
    'Confirmacion_Recordatorio',
    'Mes_Atencion',
    'Dia_Semana_Atencion'
]

# Unir variables predictoras
variables_predictoras = (
    variables_numericas + variables_categoricas
)

print("Variables numéricas:", len(variables_numericas))
print("Variables categóricas:", len(variables_categoricas))
print("Total de variables predictoras:", len(variables_predictoras))
```

SALIDA GUARDADA

```
Variables numéricas: 5
Variables categóricas: 9
Total de variables predictoras: 14
```

12. Pipeline de preprocesamiento

Fuente: SaludConex(7).ipynb · Celda(s): 46

CÓDIGO DEL NOTEBOOK

```
CELDA 46
procesamiento_numerico = Pipeline([
    ('imputador', SimpleImputer(strategy='median')),
    ('escalador', StandardScaler())
])

# Pipeline para variables categóricas
procesamiento_categorico = Pipeline([
    ('imputador', SimpleImputer(strategy='most_frequent')),
    ('codificador', OneHotEncoder(handle_unknown='ignore'))
])

# Integrar ambos procesos
preprocesador = ColumnTransformer([
    ('numericas', procesamiento_numerico, variables_numericas),
    ('categoricas', procesamiento_categorico, variables_categoricas)
])

print("Preprocesador creado correctamente.")
```

SALIDA GUARDADA

```
Preprocesador creado correctamente.
```

Imputación, estandarización y codificación de variables.

13. Comprobación de Data Leakage

Fuente: SaludConex(7).ipynb · Celda(s): 48

CÓDIGO DEL NOTEBOOK

CELDA 48

```
variables_prohibidas = [
    'ID_Reserva',
    'ID_Paciente_Anonimizado',
    'Profesional_ID',
    'Resultado_Reserva',
    'Es_Inasistencia',
    'Objetivo_Inasistencia',
    'Fecha_Solicitud',
    'Fecha_Atencion'
]

variables_detectadas = [
    variable for variable in variables_predictoras
    if variable in variables_prohibidas
]

print("Variables prohibidas encontradas:", variables_detectadas)

if len(variables_detectadas) == 0:
    print("Correcto: no se detectaron variables prohibidas.")
else:
    print("Advertencia: revisar las variables predictoras.")
```

SALIDA GUARDADA

```
Variables prohibidas encontradas: []
Correcto: no se detectaron variables prohibidas.
```

14. Separación temporal de entrenamiento y prueba

Fuente: SaludConex(7).ipynb · Celda(s): 50, 52

CÓDIGO DEL NOTEBOOK

```
CELDA 50
fecha_corte = df_model['Fecha_Atencion'].quantile(0.80)

# Separar registros anteriores y posteriores
train_df = df_model[
    df_model['Fecha_Atencion'] < fecha_corte
].copy()

test_df = df_model[
    df_model['Fecha_Atencion'] >= fecha_corte
].copy()

print("Fecha de corte:", fecha_corte.date())

print("Registros de entrenamiento:", len(train_df))
print("Registros de prueba:", len(test_df))

CELDA 52
X_train = train_df[variables_predictoras]
y_train = train_df[‘Objetivo_Inasistencia']

X_test = test_df[variables_predictoras]
y_test = test_df['Objetivo_Inasistencia']

print("Dimensiones de X_train:", X_train.shape)
print("Dimensiones de X_test:", X_test.shape)

print("Dimensiones de y_train:", y_train.shape)
print("Dimensiones de y_test:", y_test.shape)
```

SALIDA GUARDADA

```
Fecha de corte: 2026-04-06
Registros de entrenamiento: 37569
Registros de prueba: 9443

Dimensiones de X_train: (37569, 14)
Dimensiones de X_test: (9443, 14)
Dimensiones de y_train: (37569,)
Dimensiones de y_test: (9443,)
```

Fecha de corte y dimensiones de X_train, X_test, y_train e y_test.

15. Transformación y distribución de clases

Fuente: SaludConex(7).ipynb · Celda(s): 54, 56

CÓDIGO DEL NOTEBOOK

```
CELDA 54
X_train_preparado = preprocesador.fit_transform(X_train)

# Aplicar las mismas transformaciones a prueba
X_test_preparado = preprocesador.transform(X_test)

print("Datos de entrenamiento preparados:", X_train_preparado.shape)
print("Datos de prueba preparados:", X_test_preparado.shape)

CELDA 56
balance_clases = pd.DataFrame({
    'Entrenamiento (%)': (
        y_train.value_counts(normalize=True).reindex([0, 1]) * 100
    ),
    'Prueba (%)': (
        y_test.value_counts(normalize=True).reindex([0, 1]) * 100
    )
})

balance_clases.index = ['Asistencia', 'Inasistencia']

display(balance_clases.round(2))
```

SALIDA GUARDADA

```
Datos de entrenamiento preparados: (37569, 64)
Datos de prueba preparados: (9443, 64)

Entrenamiento (%)  Prueba (%)
Asistencia                82.07       81.87
Inasistencia              17.93       18.13
```

El preprocesador genera 64 columnas y se compara la proporción de clases.

16. Separación de entrenamiento y validación

Fuente: SaludConex(7).ipynb · Celda(s): 66, 71

CÓDIGO DEL NOTEBOOK

```
CELDA 66
fecha_corte_validacion = (
    train_df['Fecha_Atencion'].quantile(0.80)
)

# Crear conjunto de entrenamiento del modelo
train_model_df = train_df[
    train_df['Fecha_Atencion'] < fecha_corte_validacion
].copy()

# Crear conjunto de validación
valid_df = train_df[
    train_df['Fecha_Atencion'] >= fecha_corte_validacion
].copy()

# Mostrar resultados
print("Fecha de corte:", fecha_corte_validacion.date())
print("Registros para entrenamiento:", len(train_model_df))
print("Registros para validación:", len(valid_df))

CELDA 71
X_train_model = train_model_df[variables_predictoras]
y_train_model = train_model_df['Objetivo_Inasistencia']

X_valid = valid_df[variables_predictoras]
y_valid = valid_df['Objetivo_Inasistencia']

# Crear una copia sin ajustar del preprocesador
preprocesador_modelo = clone(preprocesador)

# Ajustar y transformar los datos de entrenamiento
X_train_model_preparado = (
    preprocesador_modelo.fit_transform(X_train_model)
)
```

```
# Transformar los datos de validación
X_valid_preparado = (
    preprocesador_modelo.transform(X_valid)
)

print("Entrenamiento:", X_train_model_preparado.shape)
print("Validación:", X_valid_preparado.shape)
```

SALIDA GUARDADA

```
Fecha de corte: 2025-10-22
Registros para entrenamiento: 30012
Registros para validación: 7557

Entrenamiento: (30012, 64)
Validación: (7557, 64)
```

4.3. Desarrollo del modelo predictivo

Con los datos preparados se inició la etapa de modelamiento. El objetivo fue comparar distintos algoritmos y determinar cuál ofrecía mejores resultados para identificar reservas con posibilidad de terminar en inasistencia.

Se trabajó con tres alternativas: un modelo de referencia, Regresión Logística y Random Forest.

Modelo de referencia (Baseline)

Se utilizó un **DummyClassifier** con la estrategia **most_frequent**. Este modelo siempre predice la clase más frecuente, que en este caso corresponde a la asistencia.

Su función fue establecer un punto de comparación. Aunque puede alcanzar una exactitud elevada debido a que la mayoría de las reservas terminan en asistencia, no tiene capacidad para identificar las inasistencias.

Regresión Logística

La Regresión Logística fue implementada como un modelo de clasificación binaria. Su función es analizar la relación entre las variables de entrada y el resultado de la reserva, generando una puntuación asociada a la posibilidad de inasistencia.

Se utilizó **class_weight='balanced'** para considerar el desbalance entre las clases. De esta manera, el algoritmo otorga mayor importancia a los errores relacionados con la clase minoritaria.

Random Forest

También se implementó Random Forest, un algoritmo que combina múltiples árboles de decisión para generar una clasificación conjunta. Esta alternativa permite analizar relaciones más complejas entre las variables.

Se configuró con 150 árboles, límite de características por raíz cuadrada, un mínimo de tres registros por hoja y ponderación de clases.

Comparación de modelos

Para seleccionar la alternativa más adecuada se utilizaron métricas como Accuracy, Precision, Recall, F1-Score y AUC-ROC.

Los resultados de validación fueron los siguientes:

Modelo	Accuracy	Precision	Recall	F1-Score	AUC-ROC
Baseline	0,821	0,000	0,000	0,000	0,500
Regresión Logística	0,594	0,231	0,544	0,325	0,611
Random Forest	0,809	0,341	0,067	0,112	0,596

Aunque el Baseline obtuvo la mayor exactitud, no detectó ninguna inasistencia. Esto demuestra por qué no es suficiente evaluar un modelo únicamente por el porcentaje general de aciertos.

Random Forest logró una mayor precisión que Regresión Logística, pero identificó una proporción mucho menor de las inasistencias reales. En cambio, la Regresión Logística alcanzó un Recall de 0,544 y un F1-Score de 0,325.

Por este motivo, se seleccionó la Regresión Logística como alternativa principal. Su mayor capacidad de detección resultaba más alineada con el propósito del proyecto: identificar una mayor cantidad de reservas que podrían terminar en inasistencia.

17. Entrenamiento de modelos predictivos

Fuente: SaludConex(7).ipynb · Celda(s): 74, 77, 80, 83, 84

CÓDIGO DEL NOTEBOOK

```
CELDA 74
modelo_base = DummyClassifier(
    strategy='most_frequent'
)

# Entrenar modelo
modelo_base.fit(
    X_train_model_preparado,
    y_train_model
)

# Realizar predicciones
pred_base = modelo_base.predict(
    X_valid_preparado
)

print("Modelo de referencia entrenado correctamente.")

CELDA 77
modelo_logistico = LogisticRegression(
    max_iter=1500,
    class_weight='balanced',
    random_state=42
)

# Entrenar el modelo
modelo_logistico.fit(
    X_train_model_preparado,
    y_train_model
)

print("Modelo de Regresión Logística entrenado correctamente.")

CELDA 80
pred_logistico = modelo_logistico.predict(
    X_valid_preparado
)

# Obtener puntuaciones de probabilidad para la clase inasistencia
prob_logistico = modelo_logistico.predict_proba(
    X_valid_preparado
)[:, 1]
```

```
# Mostrar algunos resultados
print("Predicciones:", pred_logistico[:10])
print("Puntuaciones:", prob_logistico[:10].round(3))

CELDA 83
modelo_random_forest = RandomForestClassifier(
    n_estimators=150,
    max_features='sqrt',
    min_samples_leaf=3,
    class_weight='balanced_subsample',
    n_jobs=-1,
    random_state=42
)

# Entrenar el modelo
modelo_random_forest.fit(
    X_train_model_preparado,
    y_train_model
)

print("Modelo Random Forest entrenado correctamente.")

CELDA 84
pred_random_forest = modelo_random_forest.predict(
    X_valid_preparado
)

# Obtener puntuaciones de probabilidad
prob_random_forest = modelo_random_forest.predict_proba(
    X_valid_preparado
)[:, 1]

# Mostrar algunos resultados
print("Predicciones:", pred_random_forest[:10])
print("Puntuaciones:", prob_random_forest[:10].round(3))
```

SALIDA GUARDADA

```
Modelo de referencia entrenado correctamente.

Modelo de Regresión Logística entrenado correctamente.

Predicciones: [0 0 1 0 1 1 0 0 0 1]
Puntuaciones: [0.372 0.488 0.504 0.482 0.544 0.62  0.42  0.464 0.37  0.528]

Modelo Random Forest entrenado correctamente.

Predicciones: [0 0 0 0 1 1 0 0 0 0]
Puntuaciones: [0.245 0.285 0.23  0.318 0.557 0.502 0.153 0.38  0.256 0.346]
```

18. Comparación de modelos en validación

Fuente: SaludConex(7).ipynb · Celda(s): 90

Modelo	Accuracy	Precision	Recall	F1-Score	AUC-ROC	PR-AUC
Baseline	0.821	0.000	0.000	0.000	0.500	0.179
Regresión Logística	0.594	0.231	0.544	0.325	0.611	0.261
Random Forest	0.809	0.341	0.067	0.112	0.596	0.248

4.4. Evaluación del modelo

Después de seleccionar la Regresión Logística, se realizó una evaluación más detallada para comprender su comportamiento y determinar cómo se podían utilizar sus resultados.

Matriz de confusión

La matriz de confusión permite observar cuántas predicciones fueron correctas y cuántas fueron equivocadas, separando los casos de asistencia e inasistencia.

En la validación inicial, utilizando el umbral de 0,50, se obtuvieron los siguientes resultados:

Resultado	Cantidad
Verdaderos negativos	3.754
Falsos positivos	2.447
Falsos negativos	619
Verdaderos positivos	737

En el contexto del proyecto, un falso positivo representa una reserva marcada como posible inasistencia cuando finalmente el paciente sí asiste. Esto puede generar contactos adicionales que no eran necesarios.

Por otra parte, un falso negativo corresponde a una inasistencia que el modelo no logró detectar. Este error es relevante porque representa una oportunidad preventiva que no pudo ser identificada.

Debido al objetivo del proyecto, se consideró especialmente importante reducir los falsos negativos y mejorar la capacidad de detección.

Curvas ROC y Precision-Recall

También se utilizaron las curvas ROC y Precision-Recall para complementar la evaluación.

La curva ROC permite observar la capacidad del modelo para distinguir entre las dos clases a diferentes umbrales. En validación se obtuvo un AUC-ROC de 0,611, lo que indica una capacidad de discriminación superior a una clasificación aleatoria, aunque todavía moderada.

Por otro lado, la curva Precision-Recall permite observar la relación entre la precisión de las alertas y la cantidad de inasistencias que el modelo logra detectar. El Average Precision obtenido fue de 0,261.

Estos resultados muestran que el modelo logra identificar ciertos patrones, pero todavía presenta limitaciones en la calidad de sus predicciones. Por lo tanto, sus resultados deben utilizarse como una herramienta de apoyo y no como una decisión definitiva.

Ajuste del umbral de clasificación

Inicialmente se utilizó el umbral estándar de 0,50. Sin embargo, durante la evaluación se observó que modificar este valor permitía aumentar la sensibilidad del modelo.

Se seleccionó un umbral de aproximadamente 0,453325, buscando mejorar el equilibrio entre Precision y Recall mediante el F1-Score en el conjunto de validación.

La comparación fue la siguiente:

Métrica	**Umbral 0,50**	**Umbral ajustado**
Accuracy	0,594	0,492
Precision	0,231	0,218
Recall	0,544	0,711
F1-Score	0,325	0,334

El cambio permitió aumentar el Recall desde 54,4% hasta 71,1% en validación. Esto significa que el modelo logró identificar una mayor proporción de las inasistencias reales.

La mejora tiene una contrapartida: disminuyó la precisión y aumentó la cantidad de alertas incorrectas. En términos operativos, el personal podría tener que revisar más reservas que finalmente sí terminarían en asistencia.

Evaluación final en el conjunto de prueba

Una vez seleccionado el umbral, se volvió a entrenar el modelo utilizando el conjunto completo de entrenamiento y se evaluó sobre las 9.443 reservas del conjunto de prueba, que no habían participado en la selección del modelo ni del umbral.

Los resultados finales fueron:

Modelo	Accuracy	Precision	Recall	F1-Score	AUC-ROC
Baseline	0,819	0,000	0,000	0,000	0,500
Regresión Logística	0,518	0,229	0,700	0,345	0,630

La Regresión Logística alcanzó un Recall de 0,700 en el conjunto de prueba. Esto significa que logró identificar aproximadamente el 70% de las inasistencias reales presentes en ese conjunto.

El resultado es relevante porque mantiene una capacidad de detección cercana a la observada durante la validación. Sin embargo, la precisión de 0,229 indica que una proporción importante de las alertas generadas corresponde a reservas que finalmente no terminaron en inasistencia.

Por lo tanto, el modelo presenta una utilidad potencial para priorizar revisiones, pero todavía necesita mejoras antes de considerar una implementación operacional.

Evaluación por grupos

Finalmente, se revisó el desempeño según grupos de edad y centros médicos. El propósito fue observar si el modelo mantenía un comportamiento similar en distintos segmentos.

En los grupos de edad, el Recall se ubicó entre 0,652 y 0,732. Los valores más altos correspondieron a los grupos de 45 a 59 años y de 75 años o más.

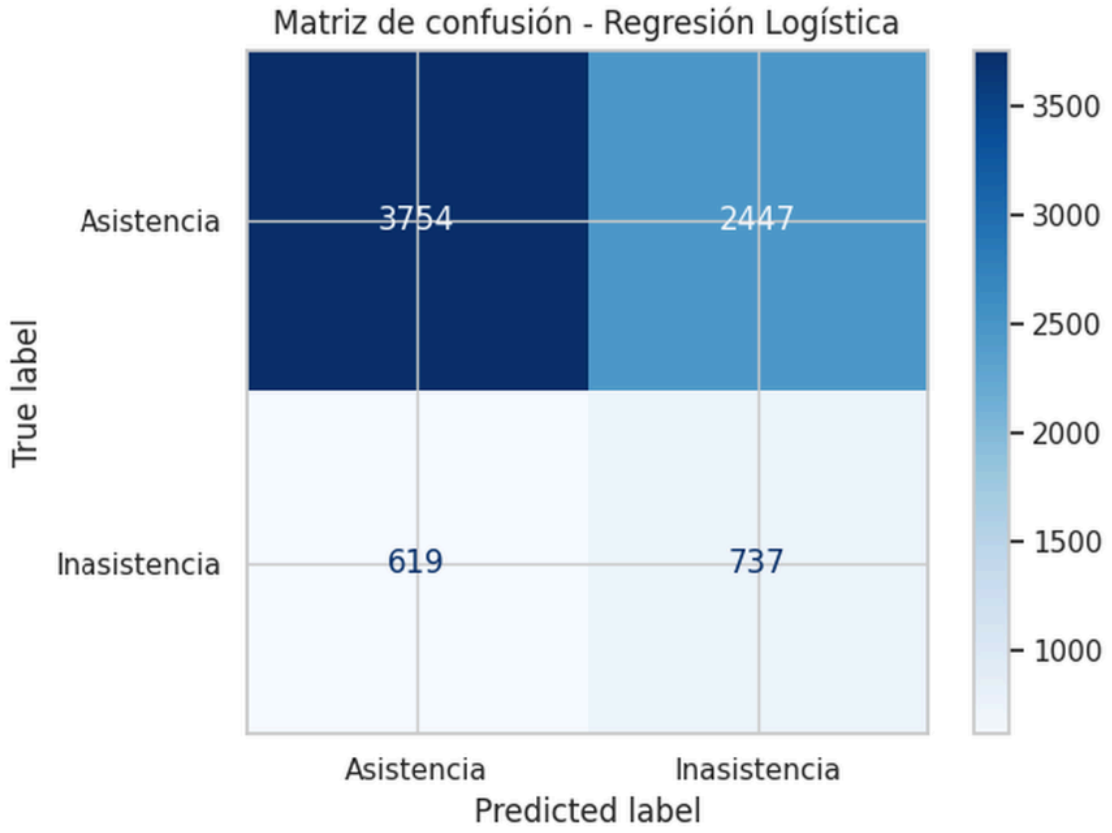
Al analizar los centros médicos, el Recall presentó una mayor variación. Ñuñoa alcanzó 0,783 y Quilicura 0,762, mientras que Puente Alto registró el valor más bajo, con 0,631.

Estas diferencias permiten identificar segmentos que podrían requerir una revisión más detallada. Sin embargo, no permiten afirmar por sí solas que exista discriminación o un sesgo sistemático. Para llegar a esa conclusión sería necesario realizar análisis

estadísticos adicionales y contar con datos reales y representativos.

Matriz de confusión de validación

Fuente: SaludConex(7).ipynb · Celda 102



20. Ejemplos de errores de clasificación

Fuente: SaludConex(7).ipynb · Celda(s): 106

	ID_Reserva	Centro_Medico	Edad	Especialidad	Dias_Anticipacion
10	RES0000011	Centro Quilicura	53	Cardiologia	7
28	RES0000029	Centro Maipu	55	Traumatologia	6
50	RES0000051	Centro Quilicura	73	Kinesiologia	9
70	RES0000071	Centro Quilicura	29	Ginecologia	11
101	RES0000102	Centro Ñuñoa	72	Cardiologia	16
122	RES0000123	Centro Santiago	85	Cardiologia	13
126	RES0000127	Centro La Florida	63	Medicina general	4
138	RES0000139	Centro Ñuñoa	15	Medicina general	12
147	RES0000148	Centro Las Condes	71	Oftalmologia	6
148	RES0000149	Centro La Florida	38	Traumatologia	2

	Resultado_Reserva	Real	Prediccion	Puntaje_Inasistencia
10	Asistio	0	1	0.504131
28	Asistio	0	1	0.544407
50	Asistio	0	1	0.527798
70	Asistio	0	1	0.728085
101	Asistio	0	1	0.508380
122	Asistio	0	1	0.533436
126	Asistio	0	1	0.678482
138	Asistio	0	1	0.674240
147	Asistio	0	1	0.660344
148	Asistio	0	1	0.546791

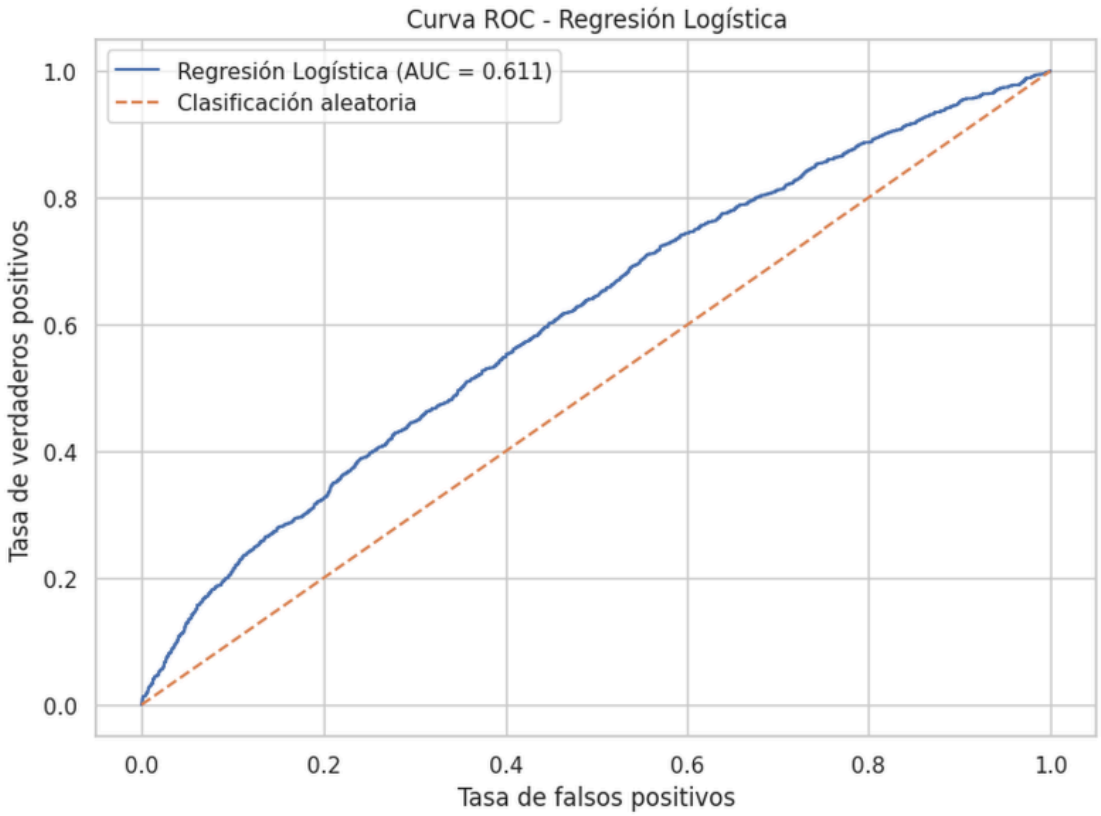
	ID_Reserva	Centro_Medico	Edad	Especialidad
19	RES0000020	Centro Ñuñoa	83	Medicina general
171	RES0000172	Centro Quilicura	6	Pediatria
197	RES0000198	Centro Santiago	42	Traumatologia
291	RES0000292	Centro Quilicura	39	Medicina general
563	RES0000564	Centro Las Condes	79	Medicina general
581	RES0000582	Centro La Florida	14	Cardiologia
711	RES0000712	Centro Maipu	40	Oftalmologia
874	RES0000875	Centro Santiago	20	Cardiologia
910	RES0000911	Centro Ñuñoa	33	Pediatria
991	RES0000992	Centro Providencia	69	Pediatria

	Dias_Anticipacion	Resultado_Reserva	Real	Prediccion
19	6	Inasistencia	1	0
171	2	Inasistencia	1	0
197	14	Inasistencia	1	0
291	6	Inasistencia	1	0
563	7	Inasistencia	1	0
581	19	Inasistencia	1	0
711	4	Inasistencia	1	0
874	7	Inasistencia	1	0
910	20	Inasistencia	1	0
991	7	Inasistencia	1	0

	Puntaje_Inasistencia
19	0.481944
171	0.439119
197	0.406220
291	0.409302
563	0.461995
581	0.462617
711	0.380095
874	0.407310
910	0.387404
991	0.392289

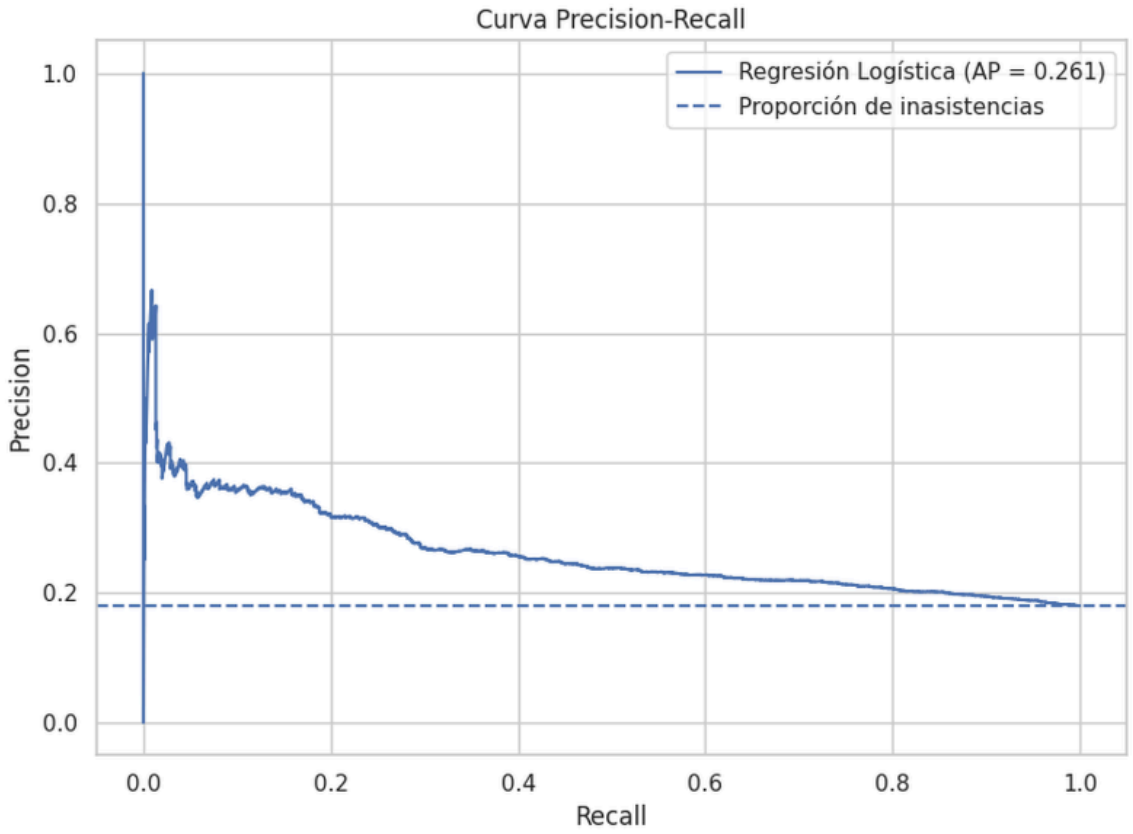
Curva ROC de validación

Fuente: SaludConex(7).ipynb · Celda 110



Curva Precision-Recall de validación

Fuente: SaludConex(7).ipynb · Celda 112



23. Selección y comparación del umbral

Fuente: SaludConex(7).ipynb · Celda(s): 115, 117

CÓDIGO DEL NOTEBOOK

```
CELDA 115
precision_valores, recall_valores, umbrales = (
    precision_recall_curve(
        y_valid,
        prob_logistico
    )
)

# Calcular F1 para cada umbral
f1_valores = (
    2 * precision_valores[:-1] * recall_valores[:-1]
    / (precision_valores[:-1] + recall_valores[:-1] + 1e-10)
)

# Encontrar el umbral que maximiza F1
mejor_indice = np.argmax(f1_valores)

umbral_optimo = umbrales[mejor_indice]

print(f"Umbral seleccionado. {umbral_optimo..4f}")
print(f"Precision: {precision_valores[mejor_indice]:.4f}")
print(f"Recall: {recall_valores[mejor_indice]:.4f}")
print(f"F1-Score: {f1_valores[mejor_indice]:.4f}")

CELDA 117
pred_valid_ajustado = (
    prob_logistico >= umbral_optimo
).astype(int)

# Comparar métricas antes y después
comparacion_umbrales = pd.DataFrame([
    {
        'Umbral': 0.50,
        'Accuracy': accuracy_score(y_valid, pred_valid_05),
        'Precision': precision_score(y_valid, pred_valid_05),
        'Recall': recall_score(y_valid, pred_valid_05),
        'F1-Score': f1_score(y_valid, pred_valid_05)
    },
    {
        'Umbral': umbral_optimo,
        'Accuracy': accuracy_score(y_valid, pred_valid_ajustado),
        'Precision': precision_score(y_valid, pred_valid_ajustado),
        'Recall': recall_score(y_valid, pred_valid_ajustado),
        'F1-Score': f1_score(y_valid, pred_valid_ajustado)
    }
])
```

```
display(comparacion_umbrales.round(3))
```

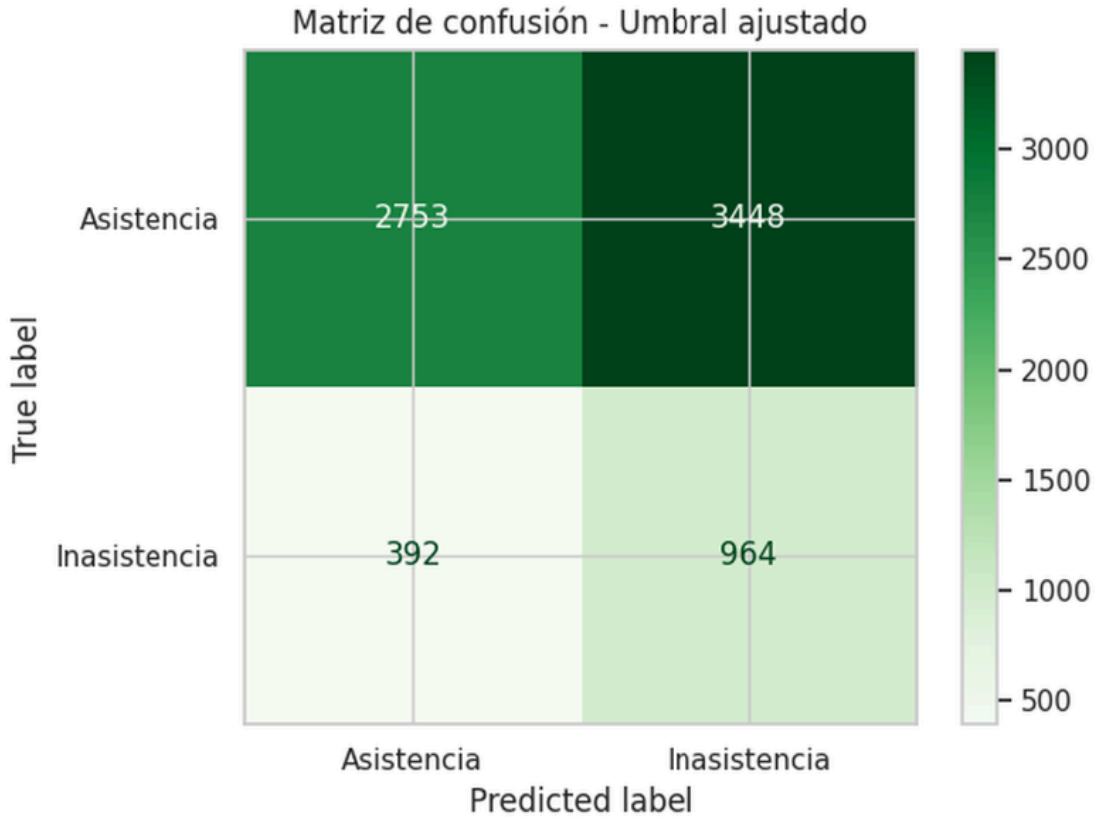
SALIDA GUARDADA

```
Umbral seleccionado: 0.4533
Precision: 0.2185
Recall: 0.7109
F1-Score: 0.3343

Umbral  Accuracy  Precision  Recall  F1-Score
0   0.500     0.594      0.231   0.544     0.325
1   0.453     0.492      0.218   0.711     0.334
```

Matriz de confusión con umbral ajustado

Fuente: SaludConex(7).ipynb · Celda 119



25. Métricas finales en el conjunto de prueba

Fuente: SaludConex(7).ipynb · Celda(s): 128

```
                                  Accuracy  Precision  Recall  F1-Score  AUC-ROC  \
Modelo
Regresión Logística Final            0.518      0.229     0.7     0.345     0.63

                                  PR-AUC
Modelo
Regresión Logística Final          0.272
```

Métricas finales de Regresión Logística sobre 9.443 registros.

26. Comparación final con Baseline

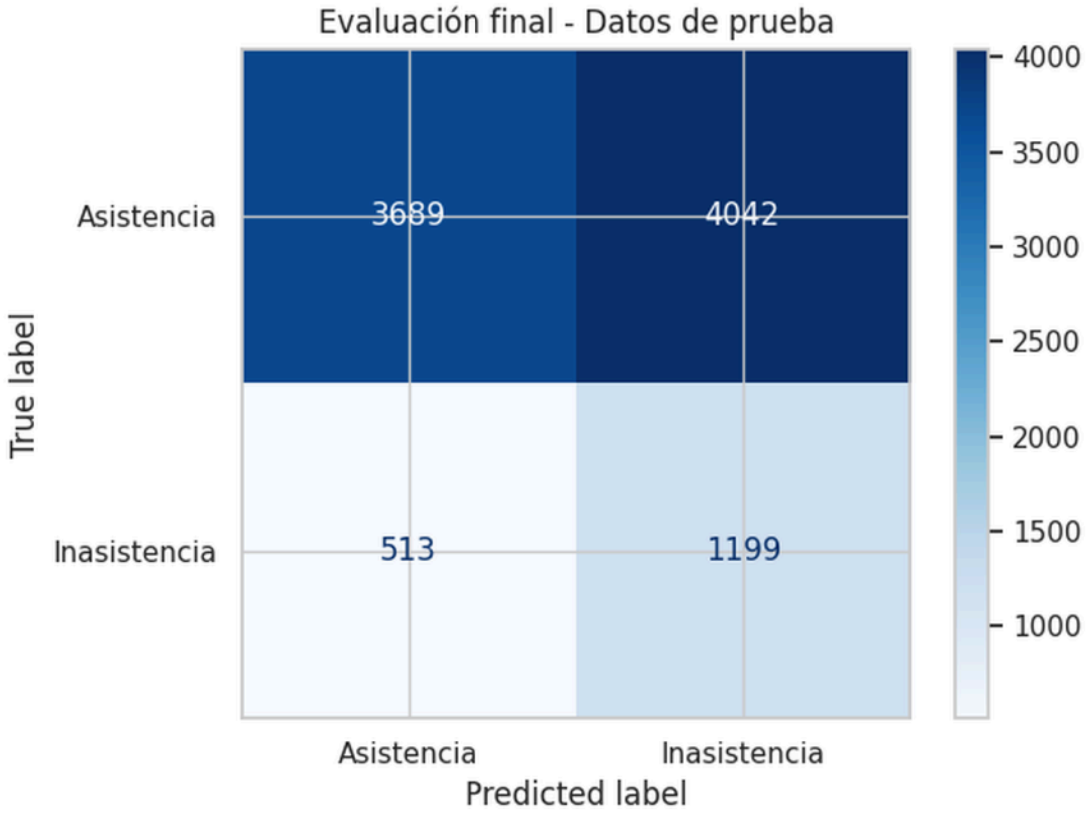
Fuente: SaludConex(7).ipynb · Celda(s): 130

Modelo	Accuracy	Precision	Recall	F1-Score	AUC-ROC	PR-AUC
Baseline	0.819	0.000	0.0	0.000	0.50	0.181
Regresión Logística Final	0.518	0.229	0.7	0.345	0.63	0.272

Comparación final de métricas en el conjunto de prueba.

Matriz de confusión final en prueba

Fuente: SaludConex(7).ipynb · Celda 132



4.5. Generación de archivos para Business Intelligence

Una vez finalizada la evaluación, se prepararon dos archivos CSV para facilitar el uso de los resultados en Looker Studio.

La finalidad fue separar la información que podría utilizarse para una gestión preventiva de aquella destinada a evaluar el comportamiento del modelo.

Archivo de predicciones operativas

Se generó el archivo SaludConex_Predicciones_Looker.csv, que contiene 9.443 registros y 13 columnas.

Este archivo incorpora información de las reservas, como centro médico, especialidad, edad, comuna, fecha y horario de atención, además de variables utilizadas para contextualizar el puntaje generado por el modelo.

Entre los campos incorporados se encuentra Puntaje_Inasistencia, que corresponde a la salida numérica del modelo, y Umbral_Utilizado, que permite identificar el punto de corte empleado para clasificar los registros.

A partir del umbral seleccionado se generaron dos categorías:

- Riesgo alto: reservas cuyo puntaje es igual o superior al umbral.
- Riesgo no alto: reservas cuyo puntaje es inferior al umbral.

En el conjunto de prueba se obtuvieron 5.241 reservas clasificadas como riesgo alto y 4.202 como riesgo no alto.

Esta clasificación permite ordenar los registros y facilitar la revisión de aquellos que presentan un puntaje más elevado.

Archivo de evaluación retrospectiva

También se generó **SaludConex_Evaluacion_Modelo.csv**, con 9.443 registros y ocho columnas.

A diferencia del archivo anterior, este conjunto incorpora el resultado real de las reservas junto con la predicción del modelo. Esto permite comparar lo que el algoritmo estimó con lo que efectivamente ocurrió dentro del conjunto de prueba.

Su utilidad principal es evaluar el desempeño del modelo mediante indicadores de aciertos, errores y resultados por segmentos.

Es importante distinguir ambos archivos: el de predicciones contiene información para organizar una posible gestión preventiva, mientras que el de evaluación se utiliza para revisar retrospectivamente el desempeño del modelo. El resultado real de una reserva no debe utilizarse como dato de entrada para predecir una cita futura, ya que solo se conoce una vez finalizada la atención.

Validaciones de los archivos

Se realizaron comprobaciones para verificar la correspondencia entre las predicciones y los registros originales. También se revisó la estructura de los archivos exportados.

El archivo de predicciones fue generado sin valores nulos o celdas vacías según las comprobaciones realizadas en el notebook. Se utilizó codificación UTF-8 con BOM para mantener una correcta lectura de caracteres especiales y se limitaron los valores decimales a seis posiciones.

Con estos archivos se dejó preparada la información necesaria para construir las visualizaciones en Looker Studio.

32. Comprobación de las predicciones

Fuente: SaludConex(7).ipynb · Celda(s): 149

CÓDIGO DEL NOTEBOOK

CELDA 149

```
print("Reservas de prueba:", len(test_df))
print("Puntuaciones generadas:", len(prob_test_final))
print("Predicciones generadas:", len(pred_test_final))

# Comprobar que los registros mantienen el mismo orden
assert test_df.index.equals(y_test.index)

# Comprobar que las cantidades coinciden
assert len(test_df) == len(prob_test_final) == len(pred_test_final)

print("Comprobación correcta. Podemos generar el archivo.")
```

SALIDA GUARDADA

```
Reservas de prueba: 9443
Puntuaciones generadas: 9443
Predicciones generadas: 9443
Comprobación correcta. Podemos generar el archivo.
```

33. Vista previa del archivo de predicciones

Fuente: SaludConex(7).ipynb · Celda(s): 152

	ID_Reserva	Centro_Medico	Especialidad	Edad	Comuna	Fecha_Atencion	Horario_Atencion	Dias_Anticipacion
46769	RES0046770	Centro Ñuñoa	Medicina general	69	La Florida	2026-06-09	14:00-16:00	8
16045	RES0016046	Centro Providencia	Medicina general	69	Ñuñoa	2026-08-22	16:00-18:00	37
36528	RES0036529	Centro Quilicura	Traumatologia	51	Ñuñoa	2026-09-14	14:00-16:00	32
2396	RES0002397	Centro Santiago	Kinesiologia	20	Puente Alto	2026-08-09	10:00-12:00	35
35794	RES0035795	Centro Santiago	Kinesiologia	75	Quilicura	2026-05-09	08:00-10:00	13
37301	RES0037302	Centro Maipu	Traumatologia	15	La Florida	2026-07-11	16:00-18:00	1
45337	RES0045338	Centro Ñuñoa	Medicina general	80	La Reina	2026-04-09	08:00-10:00	33
7195	RES0007196	Centro Puente Alto	Medicina general	30	San Miguel	2026-05-21	08:00-10:00	22
3101	RES0003102	Centro Ñuñoa	Medicina general	71	Santiago	2026-05-04	08:00-10:00	14
24778	RES0024779	Centro Providencia	Traumatologia	71	San Miguel	2026-04-25	16:00-18:00	24

	Confirmacion_Recordatorio	Puntaje_Inasistencia	Umbral_Utilizado	\
46769	Confirmado	0.880248	0.453325	
16045	No confirmado	0.878206	0.453325	
36528	No confirmado	0.869551	0.453325	
2396	Confirmado	0.868036	0.453325	
35794	Confirmado	0.862790	0.453325	
37301	No confirmado	0.862598	0.453325	
45337	No confirmado	0.846721	0.453325	
7195	Confirmado	0.839688	0.453325	
3101	No confirmado	0.837261	0.453325	
24778	No confirmado	0.831638	0.453325	

	Nivel_Riesgo	Alerta_Preventiva
46769	Alto	Priorizar revision
16045	Alto	Priorizar revision
36528	Alto	Priorizar revision
2396	Alto	Priorizar revision
35794	Alto	Priorizar revision
37301	Alto	Priorizar revision
45337	Alto	Priorizar revision
7195	Alto	Priorizar revision
3101	Alto	Priorizar revision
24778	Alto	Priorizar revision

34. Exportación del CSV de predicciones

Fuente: SaludConex(7).ipynb · Celda(s): 155

CÓDIGO DEL NOTEBOOK

```
CELDA 155
nombre_archivo = 'SaludConex_Predicciones_Looker.csv'

# Exportar a formato CSV
predicciones_dashboard.to_csv(
    nombre_archivo,
    index=False,
    encoding='utf-8-sig',
    float_format='%.6f'
)

print("Archivo generado correctamente:", nombre_archivo)
```

SALIDA GUARDADA

```
Archivo generado correctamente: SaludConex_Predicciones_Looker.csv
```

35. Exportación del CSV de evaluación

Fuente: SaludConex(7).ipynb · Celda(s): 159

CÓDIGO DEL NOTEBOOK

```
CELDA 159
evaluacion_dashboard = pd.DataFrame({
    'ID_Reserva': test_df['ID_Reserva'].to_numpy(),
    'Centro_Medico': test_df['Centro_Medico'].to_numpy(),
    'Edad': test_df['Edad'].to_numpy(),
    'Especialidad': test_df['Especialidad'].to_numpy(),
    'Resultado_Real': test_df['Resultado_Reserva'].to_numpy(),
    'Prediccion_Modelo': np.where(
        pred_test_final == 1,
        'Inasistencia',
        'Asistencia'
    ),
    'Puntaje_Inasistencia': prob_test_final,
    'Nivel_Riesgo': np.where(
        prob_test_final >= umbral_optimo,
        'Alto',
        'No alto'
    )
})

# Exportar archivo
evaluacion_dashboard.to_csv(
    'SaludConex_Evaluacion_Modelo.csv',
    index=False,
    encoding='utf-8-sig'
)

print("Archivo de evaluación generado correctamente.")
```

SALIDA GUARDADA

```
Archivo de evaluación generado correctamente.
```

5. INTERPRETACIÓN GENERAL DE LOS RESULTADOS

El desarrollo permitió construir un flujo de trabajo que transforma los registros de reservas en información que puede utilizarse para analizar y priorizar posibles inasistencias.

Uno de los principales resultados fue comprobar que la exactitud no siempre representa la mejor medida para evaluar un modelo. El Baseline alcanzó una exactitud cercana al 82%, pero no logró detectar ninguna inasistencia. Esto ocurre porque se limita a predecir la clase más frecuente.

En cambio, la Regresión Logística obtuvo una exactitud menor, pero consiguió identificar una parte importante de las inasistencias. Considerando que el objetivo de SaludConex es anticipar este tipo de eventos, el Recall tuvo mayor importancia en la selección del modelo.

El ajuste del umbral permitió aumentar la detección de inasistencias, aunque también generó una mayor cantidad de alertas incorrectas. Esta situación demuestra que la configuración del modelo no depende únicamente de sus resultados matemáticos, sino también de la capacidad operativa que tendría la organización para revisar las alertas.

Por otra parte, el análisis por edad y centro médico permitió observar diferencias en el rendimiento. Estos resultados pueden servir como punto de partida para futuras revisiones, especialmente si en una implementación real se detectan comportamientos distintos entre sucursales o grupos de pacientes.

En términos generales, el prototipo cumple con el propósito académico de demostrar cómo se puede utilizar el aprendizaje automático para apoyar una problemática de gestión. No obstante, los resultados deben interpretarse dentro de las limitaciones del conjunto sintético utilizado.

28. Evaluación por grupos de edad

Fuente: SaludConex(7).ipynb · Celda(s): 139

Grupo	Reservas	Inasistencias reales	Precision	Recall	F1-Score	AUC-ROC
0-17	1845	350	0.251	0.660	0.363	0.642
18-29	1276	250	0.252	0.652	0.363	0.635
30-44	1597	286	0.227	0.710	0.343	0.614
45-59	1571	269	0.223	0.732	0.342	0.660
60-74	1595	289	0.226	0.723	0.345	0.637
75+	1559	268	0.202	0.731	0.316	0.606

Métricas de Precision, Recall, F1-score y AUC-ROC por grupo etario.

29. Evaluación por centro médico

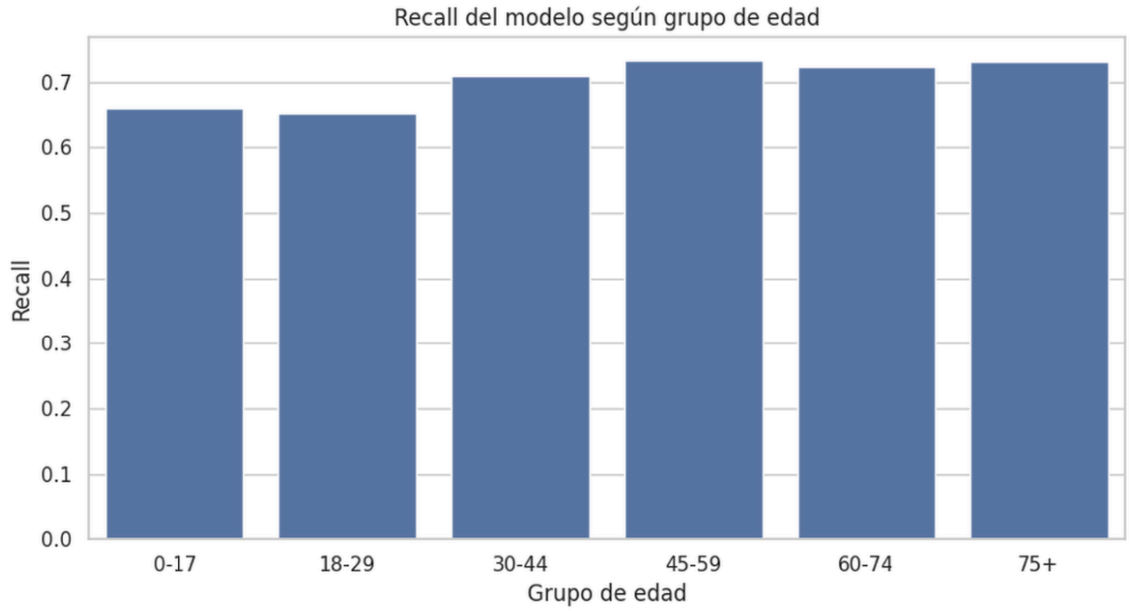
Fuente: SaludConex(7).ipynb · Celda(s): 141

Grupo	Reservas	Inasistencias reales	Precision	Recall	F1-Score	AUC-ROC
Centro La Florida	1218	197	0.201	0.675	0.309	0.615
Centro Las Condes	1007	192	0.249	0.688	0.366	0.630
Centro Maipu	1149	204	0.231	0.686	0.346	0.632
Centro Providencia	1300	226	0.207	0.695	0.319	0.623
Centro Puente Alto	1186	214	0.231	0.631	0.338	0.635
Centro Quilicura	936	185	0.255	0.762	0.383	0.659
Centro Santiago	1740	305	0.222	0.698	0.336	0.624
Centro Ñuñoa	907	189	0.252	0.783	0.381	0.621

Métricas del modelo por los ocho centros del escenario SaludConex.

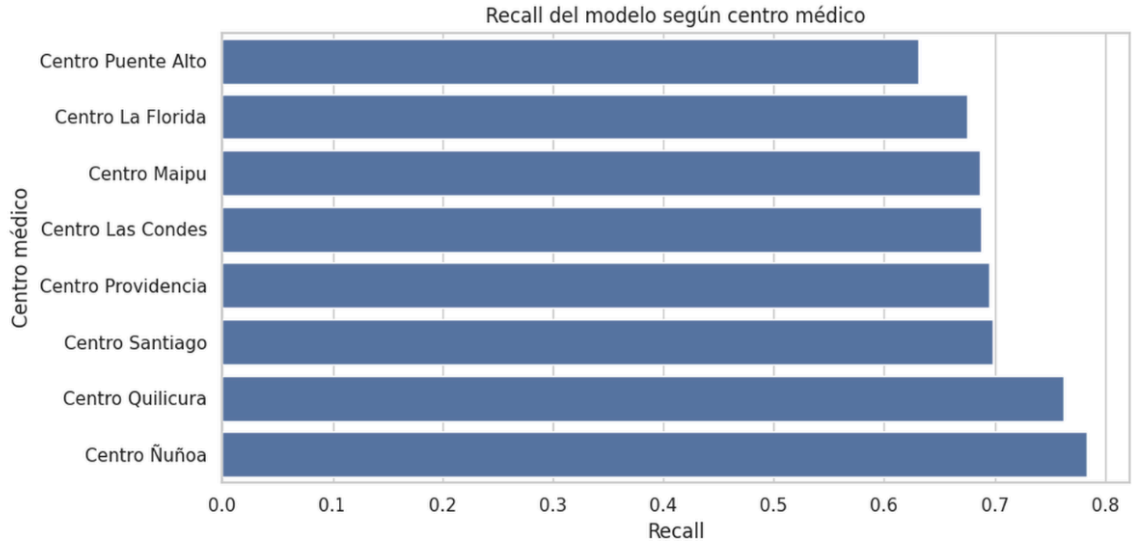
Recall del modelo por grupo de edad

Fuente: SaludConex(7).ipynb · Celda 143



Recall del modelo por centro médico

Fuente: SaludConex(7).ipynb · Celda 145



6. DESARROLLO DEL DASHBOARD EN Data STUDIO

6.1. Objetivo del dashboard

Se desarrolló un dashboard en Looker Studio con el propósito de presentar visualmente los resultados obtenidos durante la implementación del sistema predictivo SaludConex. Su objetivo principal es facilitar la identificación y priorización de reservas con mayor riesgo de inasistencia, proporcionando información que permita apoyar la toma de decisiones y orientar acciones preventivas en los centros médicos.

El dashboard permite analizar la distribución de las reservas, consultar aquellas clasificadas con riesgo alto y revisar el rendimiento del modelo predictivo. De esta manera, los resultados técnicos obtenidos en Python se presentan mediante indicadores y visualizaciones que facilitan su interpretación.

6.2. Estructura del dashboard

El dashboard se organizó en tres páginas principales, cada una orientada a un propósito específico:

- **Resumen General:** presenta una visión global de las 9.443 reservas del conjunto de prueba, mostrando su distribución por centro médico, especialidad, grupo etario, días de anticipación y nivel de riesgo.
- **Análisis de Reservas de Riesgo:** permite identificar y priorizar las reservas clasificadas con riesgo alto, incorporando indicadores, gráficos comparativos y una tabla con el detalle de las reservas.
- **Evaluación del Modelo:** presenta las métricas de rendimiento del modelo predictivo, su matriz de confusión y el comportamiento de las predicciones según centro médico y grupo etario.

Las tres páginas incorporan filtros que permiten segmentar la información según las dimensiones disponibles en cada vista. Esto facilita el análisis de distintos grupos de reservas sin necesidad de revisar los datos individualmente.

6.3. Visualizaciones e indicadores

En la página **Resumen General** se incorporaron cuatro indicadores principales: reservas analizadas, reservas de riesgo no alto, reservas de riesgo alto y umbral de clasificación. Para el conjunto de prueba se analizaron 9.443 reservas, de las cuales 5.241 fueron

clasificadas con riesgo alto y 4.202 con riesgo no alto, utilizando un umbral de clasificación de 0,453.

También se incorporaron gráficos de distribución por especialidad, volumen de reservas por centro médico, nivel de riesgo, grupo etario y días de anticipación. Estas visualizaciones permiten observar la composición de las reservas y comparar su distribución entre distintos segmentos.

La página **Análisis de Reservas de Riesgo** se orientó a la identificación de las reservas que requieren mayor atención. Presenta un total de 5.241 reservas clasificadas con riesgo alto, equivalente al 55,5 % del conjunto analizado, junto con una puntuación promedio de inasistencia de 0,48. Además, incorpora gráficos que muestran la distribución de estas reservas por centro médico y especialidad.

Como complemento, se desarrolló una tabla de detalle que permite consultar el identificador de cada reserva, centro médico, especialidad, comuna, edad, días de anticipación, puntaje de riesgo, clasificación y alerta asociada. Esta información facilita la revisión y priorización de los casos con mayores puntuaciones.

Por último, la página **Evaluación del Modelo** presenta los principales indicadores obtenidos durante la evaluación del conjunto de prueba: Recall de 70,0 %, Precision de 22,9 %, F1-score de 34,5 %, AUC-ROC de 0,630, PR-AUC de 0,272 y Accuracy de 51,8 %.

También se incorporó una matriz de confusión para visualizar las predicciones frente a los resultados reales, además de gráficos que permiten comparar el Recall por centro médico y grupo etario. Estas visualizaciones ayudan a identificar diferencias en la capacidad de detección de inasistencias entre los distintos segmentos analizados.

6.4. Evidencias del dashboard

La implementación se documentó mediante capturas de las tres páginas desarrolladas en Looker Studio, las cuales permiten observar la estructura, los indicadores, las visualizaciones y las opciones de filtrado incorporadas.

Figura X. Página Resumen General del dashboard SaludConex.

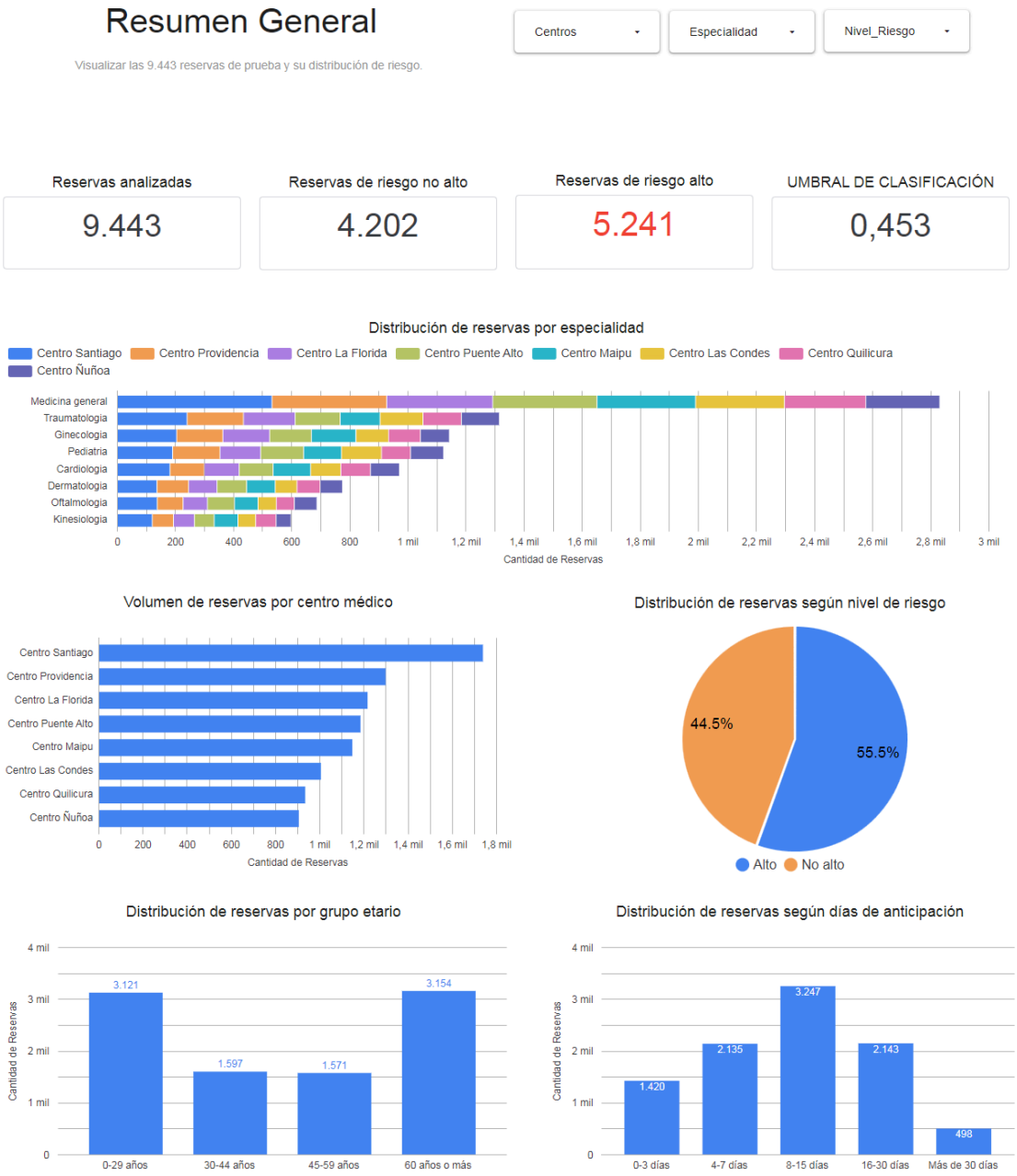


Figura X. Página Análisis de Reservas de Riesgo del dashboard SaludConex.

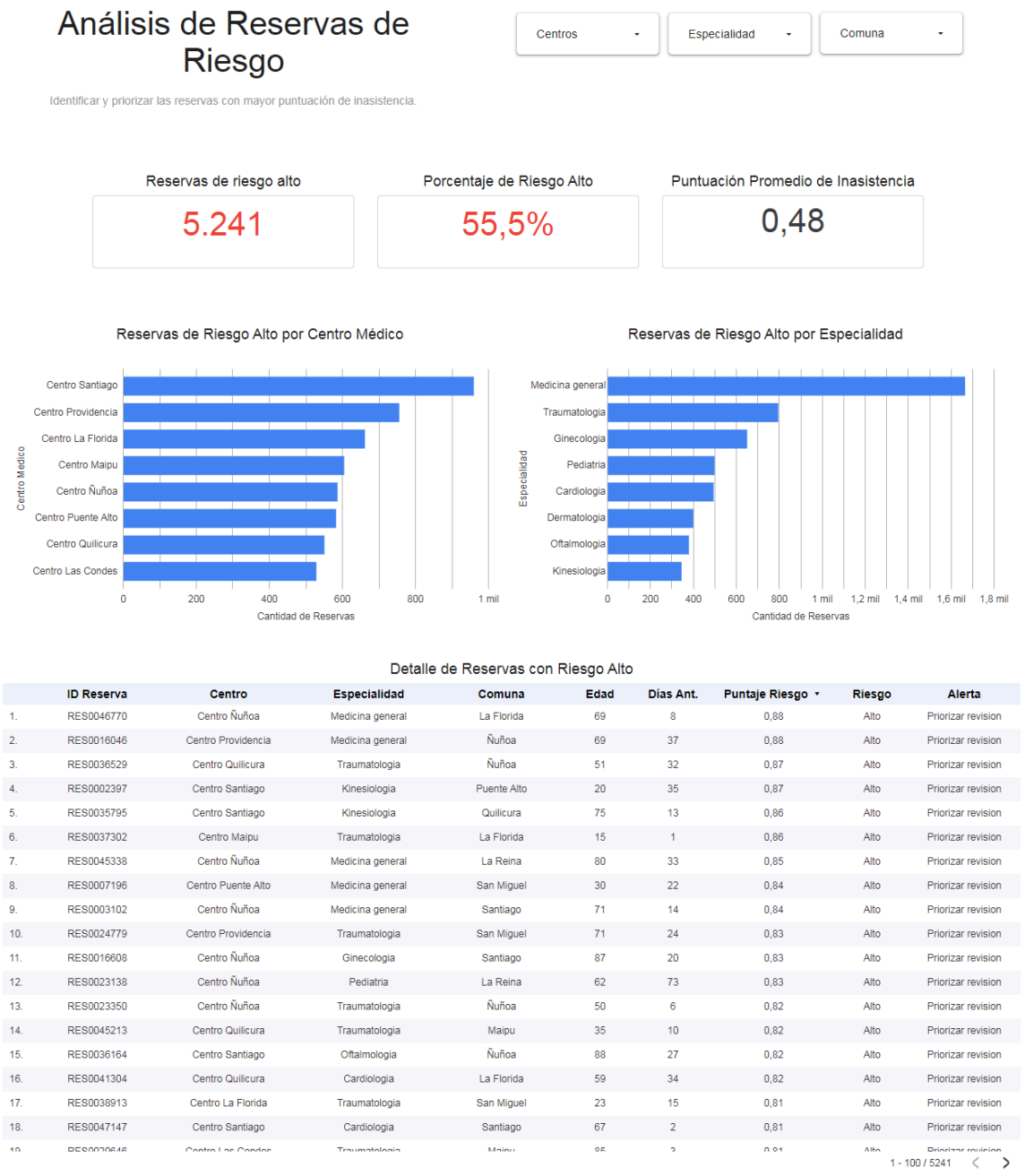


Figura X. Página Evaluación del Modelo del dashboard SaludConex.



Interpretación de los resultados:
El modelo de Regresión Logística logró detectar aproximadamente el 70% de las inasistencias reales presentes en el conjunto de prueba. Sin embargo, su precisión fue de 22,9%, lo que indica que una parte importante de las reservas clasificadas como riesgo alto terminó correspondiendo a pacientes que sí asistieron. Esto demuestra que el modelo puede ser útil para orientar la revisión y priorización de reservas, aunque todavía presenta limitaciones en la exactitud de sus alertas. Por este motivo, sus resultados deben utilizarse como apoyo a la gestión preventiva y no como una decisión automática. Los resultados corresponden a una evaluación académica realizada con datos sintéticos.

6.5. Integración de los datos

La información presentada en el dashboard proviene de los archivos generados durante la etapa de implementación y evaluación del modelo predictivo. Se utilizaron los registros correspondientes al conjunto de prueba, compuesto por 9.443 reservas, junto con las clasificaciones y puntuaciones de riesgo generadas por el modelo de Regresión Logística.

La integración de estos datos permitió construir indicadores y visualizaciones para analizar las reservas según su nivel de riesgo, centro médico, especialidad, grupo etario y

días de anticipación. Asimismo, se incorporaron los resultados de evaluación necesarios para presentar el desempeño del modelo y comparar sus predicciones con los resultados reales.

De esta forma, Looker Studio funciona como una herramienta de visualización y análisis que complementa el desarrollo realizado en Python, facilitando la interpretación de los resultados y la identificación de reservas que podrían requerir acciones preventivas.

Finalmente, es importante señalar que el dashboard corresponde a una implementación académica basada en datos sintéticos. Por lo tanto, sus resultados representan una demostración de la solución propuesta y no información proveniente de operaciones reales de una red médica.

7. Gestión y seguimiento del proyecto

La gestión del proyecto SaludConex se desarrolló mediante la planificación, coordinación y seguimiento de las actividades necesarias para implementar la solución de ciencia de datos. Para organizar el trabajo se utilizó Trello como herramienta de apoyo, permitiendo registrar las tareas, identificar responsables, establecer prioridades y controlar el estado de avance de los entregables.

La coordinación permitió distribuir las actividades entre los integrantes del equipo y mantener un seguimiento de las distintas etapas del proyecto, desde la exploración y preparación de los datos hasta el desarrollo del modelo predictivo, la construcción del dashboard y la preparación de las evaluaciones EP5 y EP6.

7.1. Organización y coordinación de las tareas

La distribución del trabajo se organizó considerando tres integrantes: Bastián Román, Ignacio Gómez y Vicente Contreras. Para facilitar la identificación de las actividades se utilizaron los códigos **BAS**, **IGN** y **VIC**, respectivamente.

Las tarjetas se agruparon por áreas de trabajo y utilizaron estos prefijos para identificar al responsable propuesto de cada actividad. La distribución registrada en el tablero contempla las siguientes responsabilidades:

Integrante	Código	Responsabilidades y actividades

Bastián Román	BAS	Coordinación y seguimiento del proyecto, organización del tablero Trello, documentación técnica de la implementación, análisis de resultados por segmentos y preparación del informe.
Ignacio Gómez	IGN	Preparación y transformación de los datos, selección de variables predictivas, tratamiento de cancelaciones, prevención de Data Leakage, separación temporal de los datos, entrenamiento y ajuste del modelo predictivo.
Vicente Contreras	VIC	Actividades de apoyo y tareas asignadas dentro de la ejecución y preparación de los entregables, de acuerdo con la distribución registrada para el integrante en Trello.

La identificación mediante códigos permitió reconocer rápidamente a qué integrante correspondía cada actividad. Asimismo, las tarjetas contemplaron información sobre la descripción de las tareas, prioridad, evidencia requerida y fechas planificadas y reales, cuando estas se encontraban confirmadas.

7.2. Seguimiento y control de las actividades

El seguimiento del proyecto se realizó mediante el tablero de Trello, donde las actividades se organizaron según su estado: pendientes, en desarrollo y completadas. Esto permitió mantener una visión general del trabajo, identificar las tareas que requerían atención y comprobar el avance de los entregables.

Entre las actividades técnicas verificadas como completadas se encuentran la revisión de los archivos CSV generados para el dashboard y la evaluación del modelo, además de la comprobación de los filtros, totales y agregaciones implementados en Looker Studio.

Esta forma de organización facilitó la coordinación del equipo y permitió mantener identificadas las actividades que aún debían finalizarse, especialmente aquellas relacionadas con la revisión final del informe y la preparación de la presentación EP6.

7.3. Comparación entre planificación y ejecución del cronograma

Para evaluar el avance del proyecto, se elaboró una comparación entre las fechas planificadas y las fechas estimadas de ejecución de las principales actividades. Debido a que no se dispone de un registro histórico completo de las fechas reales de cada tarea, las fechas de ejecución de las actividades completadas corresponden a una reconstrucción retrospectiva del avance del equipo y no a marcas de tiempo verificadas.

Actividad	Fecha planificada	Ejecución estimada o programada	Estado
Preparación y análisis exploratorio de los datos	20 al 24 de octubre	20 al 25 de octubre	Completada
Procesamiento de datos y desarrollo de modelos	24 al 29 de octubre	26 al 31 de octubre	Completada
Evaluación y selección del modelo predictivo	29 al 31 de octubre	1 de noviembre	Completada
Desarrollo del dashboard en Looker Studio	30 de octubre al 3 de noviembre	1 al 4 de noviembre	Completada
Validación de filtros, indicadores y agregaciones del dashboard	3 al 4 de noviembre	5 de noviembre	Completada
Elaboración y revisión del informe EP5	3 al 5 de noviembre	5 al 6 de noviembre	En revisión final
Preparación de la presentación EP6	6 al 8 de noviembre	6 al 8 de noviembre	Pendiente
Ensayo de la presentación de 10 minutos	8 de noviembre	8 de noviembre	Pendiente

Presentación final EP6	**Lunes 9 de noviembre**	**Lunes 9 de noviembre**	Programada

Nota: Las fechas de ejecución de las actividades completadas corresponden a estimaciones retrospectivas utilizadas para comparar la planificación con el desarrollo del proyecto. Las actividades futuras corresponden a fechas programadas y no deben considerarse como tareas ya ejecutadas.

7.4. Análisis de las diferencias entre planificación y ejecución

Al comparar el cronograma planificado con el desarrollo estimado, se observa que algunas actividades requirieron tiempo adicional respecto de lo previsto inicialmente. Esto se relacionó principalmente con la integración de los resultados del modelo en los archivos destinados a Looker Studio, la revisión de los indicadores y la validación del funcionamiento del dashboard.

La evaluación del modelo también requirió revisar diferentes alternativas y ajustar el umbral de clasificación para mejorar la detección de las inasistencias. Este trabajo fue importante para seleccionar una solución que priorizara la identificación de reservas con riesgo de inasistencia, considerando el objetivo del proyecto.

A pesar de estos ajustes, se logró completar el desarrollo del modelo, la generación de los archivos de resultados y la construcción del dashboard. Además, se verificó que los filtros, totales y agregaciones de Looker Studio funcionaran correctamente y que los archivos CSV utilizados para la visualización y evaluación mantuvieran una estructura consistente.

Las actividades que permanecen pendientes corresponden al cierre de los entregables finales: la preparación y revisión de la presentación EP6, el ensayo de exposición de 10 minutos y la presentación programada para el lunes 9 de noviembre de 2026.

7.5. Estado general del proyecto

Al momento de realizar este seguimiento, las principales actividades técnicas del proyecto se encuentran completadas. El modelo predictivo fue evaluado, los resultados fueron exportados a archivos CSV y el dashboard quedó construido y validado.

El equipo mantiene como actividades finales la preparación de la presentación EP6 y el ensayo de exposición, ambos programados antes de la presentación del lunes 9 de noviembre de 2026.

En conclusión, la planificación y coordinación mediante Trello permitió organizar las responsabilidades de los integrantes, realizar seguimiento de las actividades y comparar el avance estimado con el cronograma propuesto. Esto facilitó el control de los

entregables y la identificación de las tareas necesarias para cerrar el proyecto SaludConex.

8. DESVIACIONES, DIFICULTADES Y ACCIONES CORRECTIVAS

Durante el desarrollo del proyecto SaludConex se identificaron algunas dificultades técnicas relacionadas con el procesamiento de los datos, el entrenamiento del modelo predictivo y la integración de sus resultados en la herramienta de visualización. Estas situaciones requirieron realizar ajustes para mantener la coherencia entre los resultados obtenidos y los objetivos de la solución propuesta.

8.1. Desbalance de clases en los datos

Dificultad identificada:

Durante la preparación de los datos se observó una diferencia importante entre la cantidad de reservas correspondientes a pacientes que asistieron y aquellas asociadas a inasistencias. Esta distribución generaba un desbalance entre las clases objetivo, lo que podía provocar que el modelo favoreciera la identificación de la clase mayoritaria y presentara dificultades para detectar correctamente las inasistencias.

Impacto en el proyecto:

Esta situación podía afectar la capacidad predictiva del modelo, especialmente en la identificación de las reservas que realmente terminaban en inasistencia. Por esta razón, no resultaba suficiente evaluar el modelo únicamente mediante la exactitud (Accuracy), ya que era necesario considerar también su capacidad para detectar los casos positivos.

Acción correctiva:

Para abordar esta dificultad se incorporó el parámetro **class_weight='balanced'** durante el entrenamiento de los modelos, con el propósito de compensar la diferencia en la representación de las clases. Además, se evaluaron distintos umbrales de clasificación y se seleccionó un valor de 0,453, priorizando una mayor capacidad de detección de inasistencias.

Como resultado, el modelo final alcanzó un Recall de 70,0 %, lo que significa que logró identificar aproximadamente siete de cada diez inasistencias reales presentes en el conjunto de prueba.

8.2. Integración de los resultados con Looker Studio

Dificultad identificada:

Durante la construcción del dashboard fue necesario adaptar los archivos generados durante la implementación del modelo predictivo para utilizarlos correctamente en Looker Studio. Esta etapa requirió organizar los campos disponibles y mantener la correspondencia entre los identificadores de las reservas, las clasificaciones de riesgo y las puntuaciones generadas por el modelo.

Impacto en el proyecto:

La integración representó un desafío para mantener la consistencia entre los resultados obtenidos en Python y los indicadores presentados en el dashboard. Una interpretación incorrecta de los campos o una diferencia entre las métricas podía generar información visual que no representara fielmente el desempeño del modelo.

Acción correctiva:

Para resolver esta dificultad se organizaron los archivos de salida en formato CSV y se revisaron los campos utilizados en Looker Studio. Posteriormente, se verificó la correspondencia de las reservas, las categorías de riesgo y las puntuaciones de clasificación. También se contrastaron los indicadores del dashboard con los resultados obtenidos durante la evaluación del modelo en Python.

Este proceso permitió construir las tres páginas del dashboard: Resumen General, Análisis de Reservas de Riesgo y Evaluación del Modelo, manteniendo la coherencia entre la implementación técnica y la presentación de los resultados.

8.3. Resultado de las acciones correctivas

Las dificultades identificadas permitieron realizar ajustes tanto en la etapa de modelamiento como en la integración de los resultados. El tratamiento del desbalance de clases y la selección del umbral contribuyeron a mejorar la detección de inasistencias, mientras que la revisión de los archivos y campos permitió presentar los resultados mediante un dashboard organizado y consistente.

Estas acciones contribuyeron a completar la implementación académica de **SaludConex** y a mantener la trazabilidad entre el procesamiento de los datos, la evaluación del modelo y la visualización de sus resultados.

9. LIMITACIONES Y CONSIDERACIONES ÉTICAS

El proyecto presenta limitaciones que deben ser consideradas antes de plantear una utilización en un entorno real.

Origen de los datos

El conjunto de 50.000 registros utilizado durante el desarrollo es sintético y fue generado con fines académicos. No representa información real de pacientes, centros médicos ni operaciones de SaludConex. Por este motivo, los porcentajes y métricas obtenidos no deben interpretarse como indicadores reales de la organización.

Calidad de las predicciones

Aunque el modelo logró identificar una parte importante de las inasistencias del conjunto de prueba, su precisión fue limitada. Esto significa que varias reservas clasificadas como posibles inasistencias terminaron correspondiendo a asistencias.

Por lo tanto, el sistema no debe utilizarse como mecanismo automático para cancelar, bloquear o modificar reservas.

Interpretación de los puntajes

El campo Puntaje_Inasistencia corresponde a la salida numérica generada por la Regresión Logística. Aunque el modelo entrega valores en una escala entre cero y uno, no se realizó una validación de calibración que permita asegurar que cada puntaje represente una probabilidad real y confiable.

Por esta razón, es más adecuado interpretarlo como un puntaje de clasificación que permite ordenar las reservas según la estimación del modelo.

Uso responsable de las alertas

La clasificación de riesgo alto es una herramienta informativa. Su propósito es apoyar la priorización de revisiones y contactos preventivos, manteniendo siempre la decisión final en manos del personal responsable.

Asimismo, antes de una implementación real sería necesario evaluar periódicamente el rendimiento del modelo en los distintos centros médicos y grupos etarios, con el propósito de identificar posibles diferencias en la detección de inasistencias y evitar que determinados segmentos reciban alertas menos precisas que otros.

Privacidad y protección de la información

En una implementación real sería necesario trabajar con datos autorizados, aplicar medidas de anonimización y establecer controles de acceso. También se debería revisar periódicamente el comportamiento del modelo para detectar diferencias relevantes entre centros y grupos de pacientes.

10. CONCLUSIONES

El desarrollo del proyecto SaludConex permitió implementar un flujo de trabajo de ciencia de datos orientado a una problemática concreta de gestión: la inasistencia a consultas médicas.

A través de la exploración y preparación de los datos se logró construir un conjunto adecuado para el entrenamiento de modelos de clasificación. La utilización de una separación temporal y de un pipeline de preprocesamiento permitió mantener una metodología ordenada y reducir el riesgo de fuga de información.

Durante el modelamiento se compararon tres alternativas. La Regresión Logística fue seleccionada porque presentó una mejor capacidad de detección de inasistencias que el modelo de referencia y Random Forest, considerando especialmente el Recall y el F1-Score.

La evaluación final permitió observar que el modelo alcanzó un Recall de 0,700 en el conjunto de prueba. Esto demuestra que logró identificar aproximadamente siete de cada diez inasistencias reales presentes en ese conjunto. Sin embargo, la precisión obtenida también muestra que las alertas requieren revisión, ya que no todas corresponden a inasistencias efectivas.

La generación de los archivos CSV permitió estructurar los resultados y preparar su integración con Looker Studio. De esta manera, el trabajo no quedó limitado al entrenamiento de un algoritmo, sino que avanzó hacia la presentación de información que puede ser interpretada y utilizada para apoyar decisiones.

El desarrollo del dashboard en Looker Studio permitió complementar la solución predictiva mediante tres páginas de visualización: Resumen General, Análisis de Reservas de Riesgo y Evaluación del Modelo. A través de indicadores, gráficos comparativos, filtros y tablas de detalle, fue posible presentar la información de manera organizada y facilitar la identificación de reservas que requieren mayor atención. Esta integración permitió transformar los resultados generados en Python en información visual que puede apoyar la interpretación y priorización de las alertas.

Finalmente, es necesario destacar que el prototipo fue desarrollado con datos sintéticos. Por lo tanto, representa una demostración académica de cómo se puede abordar este problema mediante herramientas de ciencia de datos, pero no constituye todavía una solución validada para operar con pacientes reales.

Como siguiente etapa, sería necesario evaluar el modelo con datos históricos reales y autorizados, revisar su calibración, estudiar su comportamiento por segmentos y validar con usuarios administrativos si las alertas generadas aportan valor al proceso de gestión.

11. ANEXOS

Anexo A. Notebook de desarrollo

Archivo: SaludConex.ipynb

Contiene el desarrollo en Python correspondiente a la exploración, preparación, modelamiento, evaluación y generación de archivos para Business Intelligence.

Anexo B. Archivos generados

- SaludConex_Predicciones_Looker.csv: conjunto de predicciones para la preparación del dashboard.
- SaludConex_Evaluacion_Modelo.csv: conjunto de evaluación retrospectiva del modelo.

Anexo C. Evidencias de implementación

Se incorporan evidencias complementarias del desarrollo técnico y de la gestión del proyecto:

- Capturas del notebook correspondientes a la exploración, preparación, entrenamiento y evaluación del modelo.
- Capturas de la generación y exportación de los archivos CSV.
- Captura del tablero Trello actualizado, con la organización de tareas y responsabilidades del equipo.

Las capturas correspondientes al dashboard final se presentan en el apartado 6 del informe, por lo que no se duplican en este anexo.

Anexo D. Parámetros y resultados del modelo

Modelo seleccionado: Regresión Logística.

Umbral utilizado: 0,453325.

Métricas finales en el conjunto de prueba:

- Accuracy: 0,518.
- Precision: 0,229.
- Recall: 0,700.
- F1-Score: 0,345.
- AUC-ROC: 0,630.
- PR-AUC: 0,272.

Nota: Los resultados corresponden exclusivamente al conjunto de datos sintético utilizado en el desarrollo académico del proyecto.